MICA: Model-Informed Change-point Analysis

Mehdi Lotfi[1], Lars Kaderali[1*]

1 Institute of Bioinformatics, University Medicine Greifswald, Felix-Hausdorff-Str. 8, 17475 Greifswald, Germany

* lars.kaderali@uni-greifswald.de

Abstract

Change point detection is critical for identifying structural transitions in time series data. While most existing methods focus on changes in statistical properties of the data such as the mean or variance, many real-world systems are governed by dynamical models in which changes occur in model parameters. We introduce MICA, an algorithm that detects change points by minimizing the discrepancy between model simulations with a given dynamical model and observed data. The method integrates binary segmentation with a genetic algorithm to identify ~~both the timing and nature of model~~ the timing of parameter changes. Given a user-specified partition of parameters into global and segment-specific sets, MICA simultaneously estimates ~~segment-specific and global parameters alongside change points~~parameter values and change-point locations, offering enhanced flexibility and interpretability. We demonstrate its utility on synthetic datasets and real-world scenarios, including COVID-19 epidemiological modeling, under policy interventions, and the analysis of generator cooling systems in wind turbines to monitor operational status. While illustrated using differential and difference equation models, MICA is model-agnostic and applicable to any simulatable system, making it broadly useful for applications requiring accurate tracking of structural dynamics.

Author Summary

Many scientific and engineering problems involve time-series data that are generated by underlying dynamical systems. Detecting when the rules governing such a system change—so-called change points—is important for understanding interventions, faults, or regime shifts. We present MICA (Model-Informed Change-point Analysis), a computational method that detects change points by comparing observed data with simulations from a user-specified mathematical model. Unlike purely statistical methods that look for changes in averages or variances, MICA allows scientists to encode their domain knowledge through a mechanistic model and to choose which parameters are allowed to change across segments. This makes the detected breakpoints easier to interpret in the context of the underlying process. We demonstrate MICA on synthetic examples and on two real-world problems: the spread of COVID-19 in Germany, where change points may reflect policy interventions, and the operation of wind turbines, where change points can signal cooling-system faults. MICA is a flexible, offline analysis tool; it does not provide formal statistical guarantees or real-time detection, but it offers a transparent, model-aware approach to finding structural changes in complex dynamic systems.

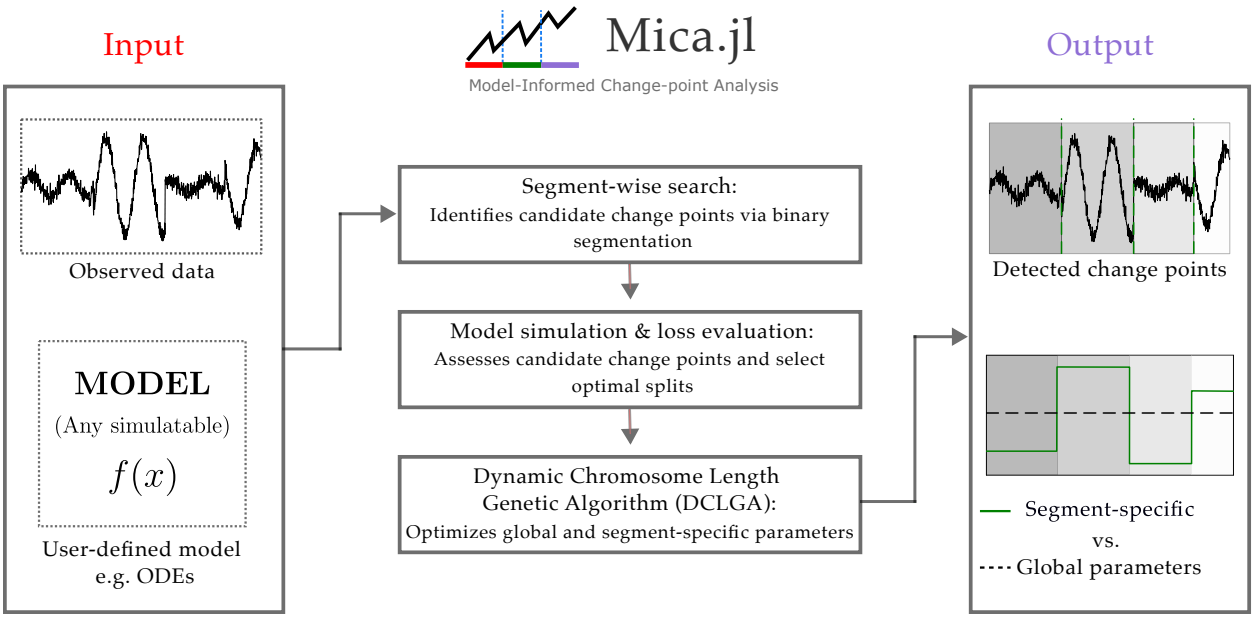

Fig 1. Graphical Abstract

Introduction

Time series models are foundational tools for analyzing dynamic systems across disciplines, capturing the sequential dependencies that drive temporal processes. From simple autoregressive models to complex dynamical systems described by differential equations, they are used extensively across scientific, engineering, and applied domains, ranging from physical and biological sciences to economics and environmental systems, to extract insights, forecast behavior, and inform decision-making.

A common challenge in time series analysis is the presence of structural transitions, or change points, where the underlying system behavior shifts due to external interventions or internal dynamics. Detecting such transitions from time series data, known as change point detection (CPD), is crucial for accurately characterizing system evolution, identifying anomalies, and improving forecasts. While some changes may coincide with known events (e.g., policy changes or hardware failures), others arise from latent processes, requiring automated and robust CPD methods.

The origins of CPD trace back to the foundational work of E.S. Page [**?**, **?**], who developed statistical techniques to detect distributional changes in sequential data. Since then, CPD has evolved into two broad classes: online methods, which detect changes in real time, and offline methods, which retrospectively segment entire datasets. Offline CPD has seen substantial growth, driven by applications in high-dimensional and nonlinear systems [**?**].

Recent advances have expanded CPD to specialized domains. In econometrics, methods based on cumulative sum statistics and score vectors capture regime shifts in volatility [**?**]. In neuroscience, modified binary segmentation and network-based approaches uncover changes in brain connectivity patterns [**?**, **?**]. Other approaches leverage graphical models [**?**], kernel methods [**?**], Bayesian frameworks [**?**], and hidden Markov models [**?**] to capture changes in statistical properties such as mean, variance, or autocorrelation.

However, most CPD methods remain focused on statistical characteristics of the data, rather than the dynamics of the system generating it. This is a key limitation when analyzing time series governed by explicit mathematical models, such as differential equations. In such systems, structural changes often manifest as shifts in model parameters, not just in observable statistics. For example, during the COVID-19 pandemic, interventions like lockdowns and vaccination campaigns changed transmission dynamics, requiring updates to subset of the epidemiological model parameters.

This insight has motivated a growing interest in ~~model-based~~ model-informed CPD. Yet many existing

methods impose restrictive assumptions, such as enforcing simultaneous changes in all parameters at each change point, or limiting the number of change points [**?**, **?**, **?**]. These simplifications can hinder both interpretability and accuracy in real-world applications.

To address these challenges, we introduce MICA (~~Model-based~~ Model-informed Change-point Analysis), a flexible algorithm designed to detect change points in time series data governed by an underlying, parametrized mathematical model. MICA frames CPD as a model selection problem, balancing goodness-of-fit with model complexity. It consists of two interacting components: a segmentation module, which proposes candidate change points using a modified binary segmentation strategy, and an optimization module, which estimates segment-specific and global parameters by minimizing the model-data discrepancy using a genetic algorithm.

A key ~~innovation in~~ feature of MICA is its support for piecewise switching models~~, where both the locations of change points~~. The user pre-specifies which parameters are global (shared across all segments) and which are segment-specific (allowed to change at change points); MICA then estimates the values of these parameters and the ~~identities of changing parameters are inferred from the data, enabling only a subset of parameters to change across segments while others remain fixed~~locations of the change points from the data. This enables fine-grained change point detection while maintaining continuity and model interpretability. The algorithm dynamically adjusts the parameter structure and leverages a cost function that penalizes over-segmentation, making it well-suited for both sparse and complex transitions.

We demonstrate MICA's effectiveness on both synthetic and real-world datasets. First, we evaluate its performance under controlled conditions in a simulation study, varying noise levels, change point counts, data lengths and so on. Second, we apply MICA to two case studies: (1) modeling the spread of COVID-19 in Germany using an ODE-based compartmental model, and (2) detecting operational status changes in wind turbine cooling systems using difference equations and Supervisory Control and Data Acquisition (SCADA) data. In both applications, MICA identifies interpretable change points that correspond to known interventions or system events, while revealing subtle transitions missed by traditional methods.

The remainder of this paper is organized as follows. Section 2 introduces the mathematical framework of MICA, including the piecewise model structure and the underlying optimization strategy. Section 3 presents results from benchmark experiments and real-world applications. We conclude with a discussion of the method's strengths, limitations, and potential future directions.

Methodology

MICA is a ~~model-based~~ model-informed change point detection framework designed for time series with underlying mathematical models. It identifies change points by minimizing the discrepancy between model simulations and observed data, while simultaneously estimating global (non-segment-specific) and local (segment-specific) model parameters. MICA supports scenarios in which only a subset of parameters changes across time, making it suitable for complex systems with large numbers of parameters, only some of which are allowed to change over time.

Although this section presents MICA in the context of ordinary differential equation (ODE) models, the framework is model-agnostic and applicable to any time series system defined by explicit mathematical formulations. This includes discrete-time systems, algebraic or hybrid models, stochastic processes, and other frameworks. The only requirement is that the model can be simulated over individual time segments, has parameters governing its behavior, and produces output that can be compared to observed data.

Problem Formulation

Notations

- Let $X = \{x_t\}_{t=1}^{n}$ represent a time series of length n. A segment $\{x_t\}_{t=a}^{b}$ is a contiguous subset satisfying $1 \leq a < b \leq n$. We denote the full dataset as $x_{1\ldots n}$ and subsegments as $x_{a\ldots b}$.
- A set $CPs = \{t_1, t_2, \ldots, t_k\} \subset \{1, 2, \ldots, n\}$, with $t_1 < t_2 < \cdots < t_k$, identifies the indices of change points, dividing the dataset into $k + 1$ segments. These points correspond to changes in some of the

parameters of the governing model.

MICA formulates change point detection as a model selection problem in the context of time series governed by parametric models. In many real-world systems, certain parameters governing system behavior may remain stable over time, while others can experience abrupt changes at unknown points due to internal dynamics or external interventions. Identifying these change points and accurately estimating model parameters across segments is critical for understanding system dynamics.

We use ordinary differential equations (ODEs) as a reference modeling framework. A classical ODE system is defined as:

$$\frac{dx}{dt} = f(x, t, p), \quad x(t_0) = x_0 \tag{1}$$

where x denotes the vector of state variables, t is time, and p is a set of fixed model parameters. This standard formulation assumes the dynamics of the system are invariant throughout the observed period.

To incorporate structural changes, we extend this to a piecewise formulation, Piecewise Switching ODEs (PSODEs):

$$\frac{dx}{dt} = \sum_{i=1}^{k+1} \mathbf{1}_{[t_{i-1}, t_i]} f_i(x, t, NSP, SP_i) \tag{2}$$

Here, k is the number of change points dividing the time horizon into $k+1$ segments. Each segment $[t_{i-1}, t_i)$ is governed by a potentially distinct parameterization of the model. NSP denotes global (non-segment-specific) parameters shared across all segments, while SP_i are segment-specific parameters local to the i-th segment.

The initial conditions for the PSODE are defined recursively. The system starts from the given initial state $x(t_0) = x_0$ for the first segment, and the state at the end of each segment serves as the initial condition for the next:

$$x(t_{i-1}) = \begin{cases} x_0, & i = 1, \\ x(t_{i-1}^-) = x(t_{i-1}^+), & i > 1. \end{cases} \tag{3}$$

This ensures continuity of the system across change points.

Also, $\mathbf{1}_{[t_{i-1}, t_i]}(t)$ is the indicator function defined as:

$$\mathbf{1}_{[t_{i-1}, t_i)}(t) = \begin{cases} 1, & \text{if } t_{i-1} \le t < t_i \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

Let $\Theta = NSP \cup SP$ denote the full set of model parameters, where NSP represents non-segment-specific parameters shared across all segments, and $SP = \{SP_1, \ldots, SP_{k+1}\}$ denotes the segment-specific parameters associated with each segment.

Formally, the objective is to estimate the set of change points $CP = \{t_1, \ldots, t_k\}$ together with the model parameters Θ by minimizing the following penalized cost function:

$$\mathcal{L}(\Theta) = \sum_{i=1}^{k+1} c\big(x_{t_{i-1}:t_i}, \hat{x}_{t_{i-1}:t_i}(NSP, SP_i)\big) + \text{pen}. \tag{5}$$

Here, $x_{t_{i-1}:t_i}$ denotes the observed data on segment i, and $\hat{x}_{t_{i-1}:t_i}(NSP, SP_i)$ is the corresponding model output obtained by simulating the piecewise dynamical system on that segment using the global parameters NSP and the local segment-specific parameters SP_i. The function $c(\cdot, \cdot)$ denotes a segment-wise discrepancy measure (e.g., squared error or RMSE), and pen is a penalty term that controls over-segmentation (e.g., a ~~BIC-based~~ complexity penalty).

Minimizing $\mathcal{L}$ jointly determines the number and locations of change points, the global parameters shared across all segments, and the segment-specific parameters, while balancing goodness of fit against model complexity.

In this study, we employed a BIC-based penalty , which provides a principled trade-off between model fit and segmentation complexity and was applied consistently across all analysesMICA supports several penalty families. The principled information-criterion penalties BIC, MDL, and AIC count the total number of free parameters, including change-point locations, and are used for the TCPD benchmark, the synthetic toy benchmark, and the real-world applications in Sections 3.3 and 3.4. In addition, a user-scalable κ-heuristic pen $= \kappa\, p\, \log(n)$ is retained for legacy sensitivity analyses; this formulation was used in the original synthetic SIR experiments reported in Section 3.2. The exact definitions of all supported criteria are given in Supplementary Section S5.

MICA Architecture

MICA performs change point detection by iteratively refining a piecewise modelthat best explains the observed time series. It begins with the entire dataset modeled . Starting from the full dataset as a single segmentand sequentially introduces candidate change points. At each step, data and model are scanned using a sliding window approach to identify positions where splitting the time series leads to a meaningful improvement in model fit. For each proposed split , parameters are estimated across all segments, and the global model error is evaluated.

New change points are accepted only if they reduce the overall discrepancy between the model simulations and observed data , it proposes candidate split positions, estimates parameters for each candidate segmentation, and accepts a new change point only when the improvement exceeds the penalty. The workflow is divided into three interacting blocks (Figure **??**). Block 1 evaluates every admissible candidate split j inside the current interval and selects the one that minimizes the penalized total loss. Block 2 accepts the candidate if it improves the current best loss by more than the penaltyvalue, effectively balancing model accuracy and complexity. , flags the position as a change point, splits the interval, and pushes the resulting sub-intervals back onto the evaluation queue. Block 3 dynamically updates the GA chromosome length so that the newly created segment receives its own parameter block. The process repeats until no candidate yields a sufficient improvement.

As new segments are introduced, the model's parameter structure dynamically adapts, ensuring consistency across the full time course.

The algorithm proceeds through iterative scans in both forward and backward directions to avoid missing any relevant structural changes. This process is stack-wise: after each newly detected change point, the resulting segments are added to a stack of segments awaiting evaluation. The forward check begins by evaluating the last segment for a potential change point. If no change point is detected, the segment is removed from consideration, and the algorithm moves to the second-to-last segment, ; this is the backward check phase. This backward evaluation continues until either a change point is found or all segments have been examined.

If a new change point is detected in the backward pass, the algorithm performs another forward and backward scan from the newly created segments to ensure that no significant change points are overlooked.

The final result is a segmented model in which each interval reflects a distinct dynamic regime, characterized by its own parameter subset. This process is illustrated in the MICA flowchart shown in Figure **??**, where red arrows denote the initial iteration without any change points, and black arrows indicate subsequent refinement steps.

As described in Algorithm **??**, the MICA framework uses a genetic algorithm to search and jointly optimize global and segment-specific parameters, while iteratively identifying change points that minimize the penalized model-data discrepancy. The pseudocode summarizes this iterative process, including candidate evaluation, model simulation, and dynamic chromosome updates, which together form the core logic behind the segmentation, optimization, and interaction modules. Although the graphical overview and chromosome encoding are presented in the context of the genetic algorithm, the Optimization Module is optimizer-agnostic: the GA backend can be replaced by derivative-free optimizers such as Nelder-Mead or BOBYQA, as well as by metaheuristic optimizers, without altering the segmentation or chromosome-update logic.

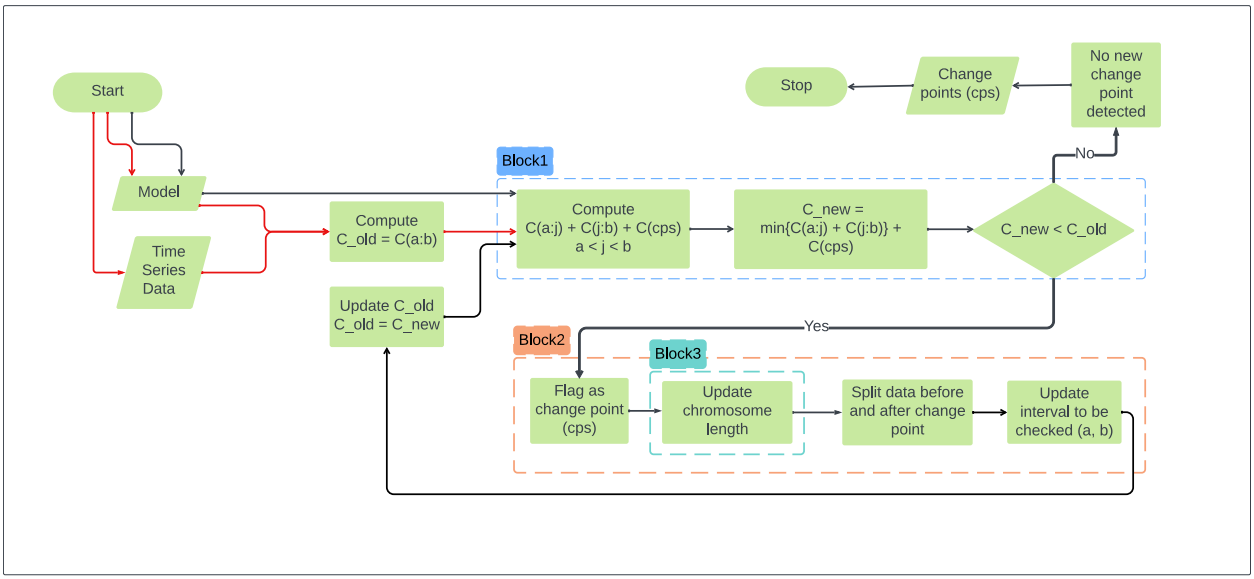

Fig 2. MICA algorithm overview. The algorithm starts with ~~an initial model~~ the full time series as a single segment and ~~dataset, and~~ iteratively proposes and validates change points. ~~The framework integrates three key modules: segmentation (~~Block 1 evaluates candidate split positions and selects the minimum-penalized-loss candidate. Block 2 ~~)~~accepts the candidate when the loss improvement exceeds the penalty, ~~optimization (Block 1)~~records the change point, and ~~chromosome updates (~~queues the resulting sub-intervals. Block 3 ~~). Red arrows indicate~~ updates the ~~initialization step without change points; black arrows show subsequent iterations~~GA chromosome by appending a new segment-specific parameter block. The process terminates when no further improvement is found.

Algorithm 1 MICA algorithm for detecting change points in dynamical systems by minimizing simulation-based loss using genetic optimization.

Input: Time series $\{x_t\}_{t=1}^{T}$, model $\mathcal{M}$, loss $\mathcal{L}$, penalty $\mathrm{pen}(\cdot)$
    Global and segment parameter counts d_g, d_s, genetic optimizer $\mathcal{G}$, minimum segment length δ, step size s
Output: Change points CP, optimized parameter set θ^*

procedure MICA
    $CP \leftarrow \emptyset$
    Initialize chromosome θ of length $d_g + d_s$
    $(L_{\text{best}}, \theta^*) \leftarrow \textsc{OptimizationModule}(CP)$
    Stack $\mathcal{S} \leftarrow \{(1, T)\}$
    while $\mathcal{S} \neq \emptyset$ **do** ▷ Main segmentation loop
        Pop (a, b) from $\mathcal{S}$
        Initialize loss list $\mathcal{L} \leftarrow [\,]$, parameter list $\Theta \leftarrow [\,]$
        for $j = a + \delta$ to $b - \delta$ step s **do** ▷ Scan for potential change points
            $CP_{\text{temp}} \leftarrow \mathrm{sort}(CP \cup \{j\})$
            $(L_j, \theta_j) \leftarrow \textsc{OptimizationModule}(CP_{\text{temp}})$
            Append L_j to $\mathcal{L}$, append θ_j to Θ
        end for
        if $\mathcal{L} \neq \emptyset$ **then**
            $(L^*, \mathrm{idx}) \leftarrow \mathrm{findmin}(\mathcal{L})$
            if $L^* < L_{\text{best}}$ **then**
                $j^* \leftarrow a + \delta + (\mathrm{idx} - 1) \cdot s$
                $CP \leftarrow \mathrm{sort}(CP \cup \{j^*\})$

$L_{\text{best}} \leftarrow L^*$, $\theta^* \leftarrow \Theta[\text{idx}]$
if $j^* - a \geq \delta$ **then**
push (a, j^*) to $\mathcal{S}$ ▷ Ensure minimum segment length
end if
if $b - j^* \geq \delta$ **then**
push (j^*, b) to $\mathcal{S}$ ▷ Ensure minimum segment length
end if
end if
end if
end while
return (CP, θ^*)
end procedure

procedure OPTIMIZATIONMODULE(CP_{temp}) ▷ Optimize model parameters given fixed change points
$K \leftarrow |CP_{\text{temp}}| + 1$
Chromosome length $\ell \leftarrow d_g + K \cdot d_s$
$\theta \leftarrow \mathcal{G}(\ell)$ ▷ Genetic algorithm evolves full chromosome
Extract global params: $\theta^{(g)} \leftarrow \theta[1 : d_g]$
Extract segment params: $\theta^{(i)} \leftarrow \theta[d_g + (i-1)d_s + 1 : d_g + i \cdot d_s]$
$L \leftarrow 0$
for $i = 1$ to K **do**
Define segment $[t_i, t_{i+1})$ using CP_{temp}
Simulate $\hat{x}^{(i)} \leftarrow \mathcal{M}(\theta^{(g)}, \theta^{(i)}, t_i, t_{i+1})$
$L \leftarrow L + \mathcal{L}(x_{t_i:t_{i+1}}, \hat{x}^{(i)})$ ▷ Accumulate segment-wise loss
end for
$L \leftarrow L + \text{pen}$ ▷ Apply penalty
return (L, θ)
end procedure

Segmentation Module

In this section, we present the Segmentation Module, a core component of the proposed change point detection framework. Inspired by classical binary segmentation, this module iteratively partitions the time series into smaller segments and identifies candidate change points, which are then passed to the Optimization Module for validation (described in the next subsection).

To tailor binary segmentation for our ~~model-based~~ model-informed approach, we modified the traditional cost function, originally based on statistical characteristics, to incorporate outputs from ODE simulations alongside observed data. This adaptation enables the algorithm to evaluate how well each segment captures the underlying system dynamics, rather than relying solely on statistical patterns.

By incorporating simulation-based model behavior into the segmentation process, MICA ensures that detected change points correspond to meaningful shifts in the system's dynamics.

Figure **??** illustrates the ~~detailed workings of the~~ segmentation process. ~~The procedure begins by analyzing the entire dataset to identify the first potential change point. A counter with a sliding window, denoted by j, identifies potential change points and the associated segments, which are then passed to the Optimization Module. The graphical representation of the counter j, which shows the index of data points at a given time point , is depicted in the upper part of Figure **??**. The process is initialized with a minimum segment length of 10 data points. During each iteration, the counter increments by one data point, progressing according to the expression $j = a + 10 : 1 : b - 10$ where a~~ Panel A shows the lifecycle of the segment pool: the full interval $(0, n)$ is pushed once; intervals are repeatedly popped, swept, and either split (if a change point is accepted) or discarded. Panel B shows a popped interval (a, b) with the counter j sweeping the admissible positions $a + \delta : s : b - \delta$. For each j, the Optimization Module returns the penalized loss L_j; the array of candidate losses is scanned to find $L^* = \min_j L_j$ and the corresponding j^*. Panel C shows the acceptance test: j^* is accepted only if the improvement $L_{\text{best}} - L^*$ exceeds the penalty.

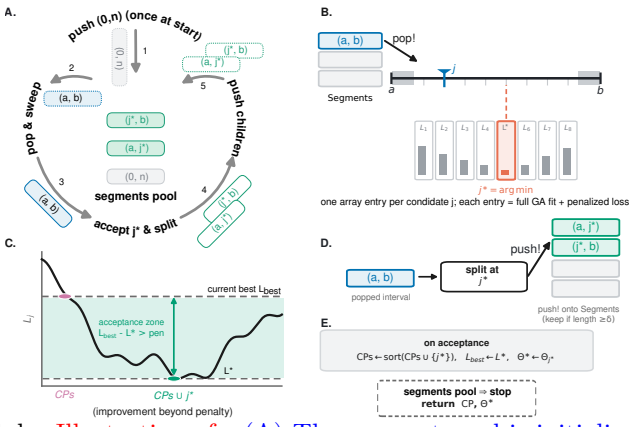

Fig 3. Segmentation Module~~: Illustration of~~. (A) The segment pool is initialized with the ~~forward-backward traversal logic using~~ full interval $(0, n)$ and processed by repeated pop-and-sweep operations. (B) For the popped interval (a, b), a ~~sliding window~~counter j sweeps admissible split positions and the penalized loss L_j is evaluated for each candidate; $j^* = \arg\min_j L_j$. (C) The candidate is accepted only if the improvement $L_{\text{best}} - L^*$ exceeds the penalty. (D) Upon acceptance, the interval is split at j^* and the two child segments are pushed back onto the pool if they satisfy the minimum-length constraint δ. (E) The change-point list, best loss, and parameter estimate are updated; the algorithm ~~incrementally updates~~ stops when the ~~counter j to identify optimal segment splits while enforcing minimum segment length~~pool is empty.

If accepted, Panel D shows the interval split at j^* and the two children pushed back onto the pool (provided each child is at least δ long). Panel E summarizes the on-acceptance update: the change point is added to the list, L_{best} and the parameter estimate are updated, and ~~b represent the start and end indices of the current segment. Initially, a is set to 0 and b to n, the total length of the given dataset. Enforcing this minimum segment length constraint ensures that each segment contains sufficient data to enable accurate parameter estimation during the optimization stage. This condition guarantees that each partition contains at least 10 time points, which is essential for accurate parameter estimation in the Optimization Module. For example, fitting ODEs with an insufficient number of data points would pose significant challenges.~~

~~During the first iteration (lower part of Figure **??**), the Optimization Module performs two tasks: (a) it identifies the optimal position for j that minimizes the total loss between the observed dataset and the model, designating this position as cp_1; and (b)it updates the interval for the counter j. Once the first change point is identified, the algorithm proceeds with subsequent iterations by fixing the initial change point and focusing the counter on the last segment created by this change point to search for the next potential change point. This process is referred to as the forward check.~~

~~In subsequent iterations (2-7), after identifying the first change point,~~ the algorithm continues ~~using the forward check (indicated by light blue arrows in Figure **??**) to Explore additional potential change points until it reaches the end of the dataset. In each iteration, the algorithm focuses on the most recent segment created by previously detected change points, leaving the remaining segments for later analysis. Any segments not immediately checked are deferred for subsequent analysis. Once the forward check is completed, the algorithm initiates a backward check (iterations 8-11). During the backward check, the algorithm revisits previously skipped segments to identify any additional change points that may have been missed during the forward check. For example, in iteration 7, since no change point was identified between cp_3 and the end of the dataset, the algorithm places the counter between cp_2 and cp_3 (iteration 8), resulting in the identification of cp_4 (iteration 9). In iteration 10, it is assumed that there is no change point between cp_4 and cp_3, and also between cp_2 and cp_4; otherwise, a forward check would have been required for the former case and a backward check for the latter. The detailed computations, particularly the cost calculations for a given interval,~~ until the pool is empty.

The minimum segment length δ and step size s are user-defined. They ensure that each partition contains enough data for reliable parameter estimation while controlling the resolution of the candidate

sweep. The detailed cost computations are elaborated in the Optimization Module subsection.

~~In summary, the Segmentation Module in MICA generates potential change point locations, which are then passed to the subsequent Optimization Module for further refinement and validation.~~

Interaction Module

The Interaction Module (Figure~~??~~) ~~serves as the central component that~~ ~~??~~) coordinates the Segmentation and Optimization Modules ~~to ensure coherent and accurate change point detection. Once potential change points are identified by the Binary Segmentation (BS) module, the dataset is divided into segmentsaccordingly. For each segment, the system of ordinary differential equations (ODEs) is solved over the segment's time span, producing simulated outputs that represent the system's expected behavior during that interval. To maintain dynamic consistency across the time series, the~~ through a four-step cycle. (1) *Segment Division:* candidate change points split the time series into segments. (2) *System Solving:* the model is simulated over each segment, using the final state of ~~each segment 's solution is used~~ one segment as the initial condition for the next ~~segment. Interaction Module: This module coordinates the other two modules, namely the segmentation and optimization modules, through four steps. A detailed explanation of each step is provided in the text.~~

~~Subsequently, an objective function (defined in Equation ??) is evaluated by comparing~~ to maintain continuity. (3) *Objective Function Evaluation:* the simulated outputs ~~to~~ are compared with the observed data ~~within each segment. This function quantifies the discrepancy between the model and the data and also incorporates a regularization mechanism that discourages overly complex models, ensuring that the identified change points are both accurate and interpretable.~~ through the penalized loss in Equation ??. (4) *Change-points List Updating:* accepted change points are recorded, the segment pool is updated, and the cycle repeats. This closed loop ensures that segmentation decisions are driven by the model-data discrepancy rather than by statistical summaries alone.

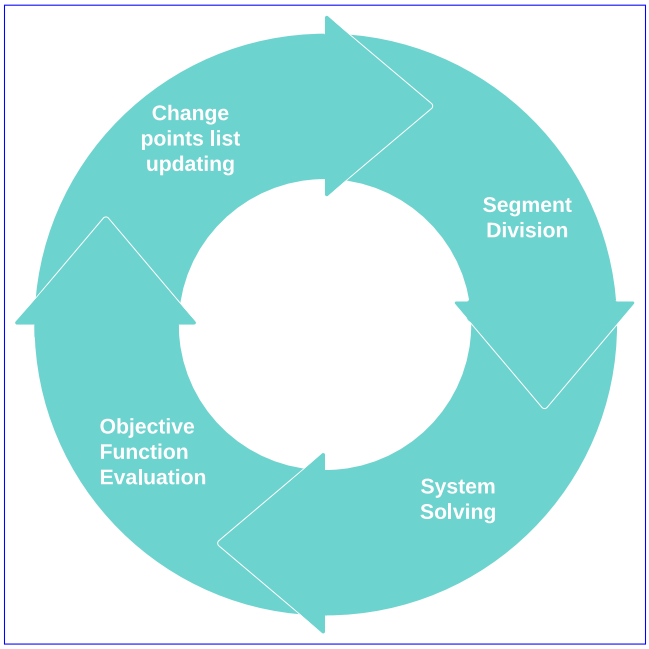

Fig 4. Interaction Module. The Segmentation and Optimization Modules are coordinated through a four-step cycle: Segment Division, System Solving, Objective Function Evaluation, and Change-points List Updating.

By combining the initial segmentation ~~from the BS module with a rigorous ODE-driven~~ with a model-driven optimization framework, this module ~~guarantees~~ ensures that the final set of change points is ~~both robust and meaningful, capturing~~ robust and captures genuine shifts in the system ~~'s behavior. The~~

~~two-step process leverages the strengths of both modules, resulting in a comprehensive and data-driven approach to ODE-based change point detection.~~ behavior.

Optimization Module

The Optimization Module takes as input the potential change points identified by the Segmentation Module and uses them to partition the dataset into corresponding segments. For each segment, the module solves the specified system of ordinary differential equations (ODEs) over the appropriate time interval, using the segment's observed data and temporal boundaries. The simulated outputs from the ODE system are then incorporated into an objective function that quantifies the fit between the model and the empirical data within each segment, ensuring a detailed and accurate assessment of model performance.

Given the interdependence of segments introduced by change points, the objective function is computed cumulatively. At each iteration of change point detection, the total cost is calculated as the sum of individual costs from all segments, both those established in previous iterations and those defined by new candidate change points. This cumulative approach is essential for capturing the global impact of adding or refining change points, allowing the optimization process to evaluate whether the introduction of a new change point improves the model's overall explanatory power. By considering the entire dataset in this aggregated manner, the algorithm effectively identifies change points that correspond to genuine shifts in the underlying system dynamics.

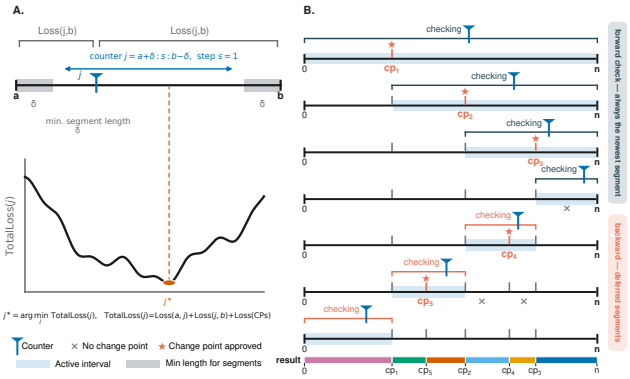

Fig 5. ~~The change point detection process within a segment during an iteration of the overall procedure~~Optimization Module. ~~Initially~~(A) Within a popped interval (a, b), the ~~start~~ counter j sweeps from $a + \delta$ to $b - \delta$ and ~~end of~~ the ~~dataset are considered, with a counter representing the first potential change point~~total penalized loss TotalLoss(j) is evaluated for each candidate. ~~As the process progresses, detected change points define new segments. Costs associated with different values are computed and stored.~~ The minimum ~~cost~~ $j^* = \arg\min_j$ TotalLoss(j) is selected and ~~its index are identified and,~~ accepted if ~~inserting this change point reduces~~ the ~~overall cost, it is designated as a new change point~~loss improvement exceeds the penalty. (B) The forward scan progressively checks the most recently created segment~~is then split~~, ~~and operations are applied to~~ while the ~~new~~ backward scan revisits earlier segments that were deferred. ~~The algorithm stops when there~~ Approved change points (stars) are recorded; intervals with no ~~more segments to investigate~~improvement (crosses) are discarded.

Figure **??** illustrates the Optimization Module~~, which is responsible for detecting change points within a given segment during each iteration of the broader change point detection process. Our method for identifying change points in time series data relies on an iterative segmentation strategy that locates time points where the system dynamics undergo significant changes. To efficiently manage which segments are yet to be analyzed, we maintain a dynamic list that is updated throughout the procedure. The implementation of this is inspired by Julia's built-in `push!` and `pop!` commands, which are commonly used for manipulating elements in arrays or stacks. Specifically, `push!` appends a new segment to the list, while `pop!` removes and retrieves the most recently added segment. This stack-based mechanism allows the algorithm to systematically explore and process each segment, ensuring thorough coverage of the dataset in the search for change points.~~

In the **initial step**, the algorithm begins by preparing the core data structures required for change point detection. A variable (`Segments`) is initialized to store the segments to be analyzed, starting with a single segment that spans the entire dataset, from index 0 to n, where n is the dataset length. This initial segment is added to the list of intervals scheduled for processing. Simultaneously, another variable (`CPs`) is prepared to keep track of the change points identified during the process. This list is initially empty and grows as the algorithm iteratively uncovers new points of change. In the **core step**, the algorithm retrieves the most recently added segment, denoted (a, b), from the list of segments awaiting analysis. This represents the current interval under investigation. In the first iteration (Case 1 in Figure ??), a and b correspond to the start and end of the dataset. As change points are progressively discovered, subsequent iterations operate on smaller intervals defined by previously identified points. As explained in Segmentation Module section, a counter j is introduced within this segment to evaluate potential positions where a change might have occurred. To maintain the reliability of parameter estimation, the counter is constrained to examine positions that yield at least δ data points in each resulting subsegment. Thus, j takes values from $a + \delta$ to $b - \delta$, where $\delta = 10$ in our implementation and the step size is set to $s = 1$. These values can be adjusted depending on the specific characteristics of the problem. For each candidate j, the algorithm computes the total cost of dividing the segment at that point. This cost includes the .
Panel A shows the sweep inside a popped interval (a, b). The counter j ranges from $a + \delta$ to $b - \delta$ with step size s. For each j, the total penalized loss is computed as the sum of the losses for the proposed subsegments (a, j) and (j, b), along with the cumulative cost associated with segments formed by loss on (a, j), the loss on (j, b), the loss associated with previously accepted change points. These values are stored in a temporary list of total costs. Case 2 in Figure ?? illustrates this process in action: after detecting several change points (e.g., cp_1 through cp_3), the algorithm examines a smaller subinterval, sweeping the counter j to test potential split locations. The algorithm then selects the value of j that results in the minimum cost (j^*). If the newly computed minimum cost is lower than that of the previous configuration, the associated j is confirmed as a new change point. This value is appended to the growing list of detected change points. In the **update step**, once a new change pointcp is validated, the current segment (a, b) is divided into two subsegments: (a, cp) and (cp, b). These new intervals are then queued for further evaluation, allowing the algorithm to recursively analyze increasingly refined sections of the dataset. The process continues iteratively: at each step, segmentsare retrieved from the queue, analyzed, and if warranted, further subdivided based on newly discovered , and the penalty. The minimum $j^* = \arg\min_j \text{TotalLoss}(j)$ is selected and passed to the acceptance test.

Panel B shows the resulting forward/backward schedule. The *forward check* always examines the newest segment created by the most recent change point; if no change point is found, the segment is discarded and the algorithm performs a *backward check* on previously deferred segments. Approved change points are marked with stars, and intervals that yield no improvement are marked with crosses. The bottom bar shows the final ordered set of detected change points. The algorithm halts when there are no more segments left to examine, specifically when the listof segments is empty, signaling that all meaningful structural shifts in the data have been detected.

In summary, the algorithm dynamically manages the segmentation process through the use of segment insertion and removal operations, iteratively

The stack-based management of segments is inspired by Julia's built-in `push!` and `pop!` operations: `push!` appends a new segment to the list, while `pop!` retrieves the most recently added segment. This mechanism lets the algorithm systematically explore and process each interval, refining the detection of change points by focusing on progressively smaller intervals segments until no further division is warranted.

Chromosome Encoding and Update

In the Optimization Module, a Dynamic Chromosome Length Genetic Algorithm (DCLGA) is employed to minimize an objective function that quantifies the discrepancy between the model simulations and the observed time series data. As visualized in Figure?? the algorithm operates on a fixed-length chromosome that encodes both global (segment-independent) parameters, denoted as $a_1, a_2, \ldots, a_m$, and model-data discrepancy. As shown in Figure ??, the chromosome encodes global non-segment-specific parameters (NSP, pink blocks) followed by segment-specific parameters, denoted as $b_1, b_2, \ldots, b_n$, where m parameter

blocks (SP, blue/orange/green blocks). At iteration k (i.e., after $k-1$ accepted change points), the chromosome length is $\ell = d_g + k \cdot d_s$, where d_g is the number of global parameters and ~~n~~ d_s is the number of parameters ~~specific to each~~ per segment.

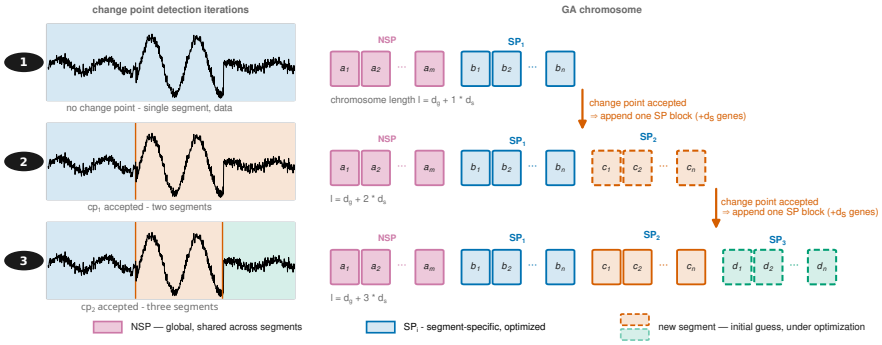

Fig 6. Dynamic Chromosome Length Genetic Algorithm (DCLGA). ~~The top row illustrates three iterations of the~~ (Left) As change ~~point detection process applied to a toy dataset generated from a system of ODEs. Each iteration highlights the number of detected change~~ points are accepted, the ~~corresponding segmentation~~ time series is split into an increasing number of ~~the data, and the solution of the model, which incorporates both segment-specific and segment-independent parameters~~segments. (Right) The ~~bottom row depicts the mechanism for updating the~~ GA chromosome ~~structure within the genetic algorithm as new~~ is extended dynamically: it begins with global (NSP) parameters followed by one segment-specific (SP) block; each accepted change ~~points are introduced~~point appends an additional SP block. Blocks corresponding to previously accepted segments remain fixed, while the newest block is optimized.

Initially, ~~in the absence of any~~ with no change points, the ~~entire time series is treated as a single segment. The genetic algorithm then optimizes the parameter set corresponding to this configuration. A counter j is introduced to identify the first change point (as detailed in the Segmentation and Optimization Modules). Once~~ chromosome contains only the NSP block and one SP block. When a change point is ~~detected, the time series is split into two segments. The chromosome is accordingly extended to include a second set of~~ accepted, a new SP block is appended; previously accepted blocks are kept fixed as optimized segment-specific parameters, ~~denoted as $c_1, c_2, \ldots, c_n$, while the global parameters $a_1, \ldots, a_m$ remain unchanged and are still positioned at the beginning of the chromosome.~~

~~The change point detection process continues iteratively. After the first change point is established, the algorithm proceeds to search for a second change point using a chromosome of the same current length. When the second change point is identified, the chromosome is again expanded to accommodate a third segment with its own parameters $d_1, d_2, \ldots, d_n$, maintaining the same structure.~~

~~At each iteration, the chromosome comprises three main components: (1) the global parameters shared across all segments, (2) the optimized parameters for previously identified segments, and (3) the initial guesses for the parameters of the newly proposed segment. This evolving structure allows the genetic algorithm to efficiently adapt the model to increasingly complex segmentations while preserving continuity and interpretability of parameter roles~~while the newest block is initialized and optimized. This dynamic extension lets the GA adapt to an increasing number of segments without re-optimizing earlier parameters from scratch.

Implementation

We provide an open-source Julia package for MICA, supporting modular model definitions and user-defined loss functions. The initial release includes built-in support for ODE-based models and difference equations. The package is extensible to custom mathematical formulations and includes tools for model simulation and parameter configuration.

The default evolutionary optimizer used by the DCLGA is a real-coded genetic algorithm with the hyperparameters listed in Table **??**. A lighter variant (population size 80, uniform-ranking selection size 10)

is used for problems with fewer than 20 parameters. The optimizer backend is interchangeable; users may alternatively supply derivative-free optimisers such as Nelder-Mead or metaheuristic optimisers.

Table 1. Default genetic-algorithm hyperparameters in `Mica.jl`.

Hyperparameter	Default value
Population size	150
Selection	uniform ranking (top 20)
Crossover	MILX(0.01, 0.17, 0.5)
Mutation	Gaussian ($\sigma = 0.0001$)
Mutation rate	0.3
Crossover rate	0.6
Generations	200

Comprehensive documentation and tutorials are available on the MICA project website (`https://changepointdetection.com`), the GitHub repository (`https://github.com/Mehdilotfi7/MICA`), and the institute's official software page (`https://wordpress.kaderali.org/software/`).

Results

In this section, we evaluate MICA on synthetic datasets and in real-world applications. We begin by evaluating the algorithm on synthetic datasets generated from ordinary differential equations (ODEs) under various controlled conditions in a simulation study, as described in the subsequent subsection. These synthetic scenarios enable a rigorous assessment of the algorithm's performance, specifically its accuracy in detecting change points, robustness to noise and parameter variations and run time, as true change points and parameters are known.

All simulations and benchmarking experiments were conducted on a Lenovo ThinkStation P2 Tower equipped with an Intel ® Core ™ Core i9-14900K processor (32 threads), 64 GiB RAM, and an NVIDIA GeForce RTX ™ 4060 GPU. The system ran Ubuntu 24.04.3 LTS (64-bit) with Linux kernel 6.14.0-29-generic and GNOME 46 (X11).

Following the synthetic evaluation, we demonstrate the algorithm's applicability to real-world datasets in two distinct domains. The first application involves change point detection in epidemiological model simulations, aimed at analyzing the effects of public health interventions on the spread of COVID-19 in Germany. This case study addresses two key objectives: first, to identify the timing of change points corresponding to events such as policy interventions and behavioral shifts; and second, to quantify the resulting impact on model parameters governing disease transmission. By pinpointing when these changes occurred and how they affected system dynamics, we gain insights into the effectiveness of different interventions. The second application focuses on operational data from wind turbines and a generator cooling system, showcasing the algorithm's utility in the renewable energy sector. In this context, the algorithm is employed to detect operational changes, thereby supporting condition monitoring and predictive maintenance strategies.

MICA in Synthetic Datasets

To assess the performance of MICA, we first apply it to synthetic time series data generated using the SIR (Susceptible–Infectious–Recovered) model, a classical compartmental framework in epidemiology that describes the flow of individuals through three states: susceptible to infection, actively infectious, and recovered (and thus immune). This model captures the core dynamics of epidemic spread and is a widely used approach to model infectious diseases. In our analysis, only the time series of the infectious compartment is used in the objective function for parameter estimation, reflecting a common scenario where limited observational data is available. Our aim here is to test the MICA's ability to detect

structural changes in model parameters, specifically, changes in the transmission rate (β) while holding the recovery rate (γ) constant.

This setup reflects realistic scenarios in which public health interventions, behavioral changes, or environmental factors can alter transmission dynamics, while recovery rates tend to remain stable over time. The temporal evolution of the disease is governed by the following system of coupled ordinary differential equations (ODEs):

$$\begin{aligned} \frac{dS}{dt} &= -\beta \frac{SI}{N} \\ \frac{dI}{dt} &= \beta \frac{SI}{N} - \gamma I \\ \frac{dR}{dt} &= \gamma I \end{aligned} \tag{6}$$

where $S(t)$ denotes the number of susceptible individuals at time t, $I(t)$ denotes the number of infectious individuals at time t, $R(t)$ denotes the number of recovered individuals at time t, β is the transmission rate of the disease, which is assumed to vary by segment, γ is the recovery rate, considered constant across segments, and N represents the total population size, given by $N = S(t) + I(t) + R(t)$.

Experimental Design and Criteria

To evaluate the effectiveness of MICA, we generate synthetic (toy) datasets based on a comprehensive set of criteria, as detailed in Table **??**. The algorithm was then applied to each generated dataset. The evaluation criteria include variations in the number of change points, dataset lengths, types and levels of noise, and penalty values. For each specific combination of these parameters, a corresponding dataset is generated to test the robustness and accuracy of the algorithm.

Evaluation Criterion	**Values**
Number of Change Points	{1, 2, 3}
Data Lengths	{70, 130, 160, 200, 250}
Noise Types	{"Gaussian", "Uniform"}
Noise Levels	{1.0, 10.0, 20.0}
Penalty Coefficient (κ)	{0.0, 1.0, 10.0, 20.0, 30.0, 100.0}
β Values	{0.00009, 0.00014, 0.00025, 0.0005}
γ Value	0.7
Change Points Positions	{50.0, 100.0, 150.0}

Table 2. Experimental design and evaluation criteria for change point detection. This table summarizes the experimental design used to assess the robustness of the proposed change point detection method across a range of conditions, including the number and locations of change points, dataset lengths, noise types and levels, and penalty values. Synthetic datasets are generated according to these criteria and used to systematically evaluate detection accuracy and computational performance.

The toy dataset is generated using the SIR model equations described in Equations **??**, incorporating two change points located at time indices 50 and 100, which divide the dataset into three distinct segments. Segment-specific parameter values for the transmission rate β (while keeping the recovery rate γ fixed) are assigned according to the configuration specified in Table **??**. Additionally, noise sampled from a uniform and gaussian distributions are added to each time point to simulate real-world measurement variability.

It is important to note that not all theoretical combinations of criteria are valid. Specifically, a combination is considered invalid if the specified locations of change points exceed the given data length. For example, if the data length is 100, it is not feasible to have a change point at position 150, as the data cannot accommodate such a change point. Therefore, we excluded such invalid combinations from our analysis. This explains why some plots are missing in the visualization of the benchmarking results. In other words, the positions of change points must be within the range of the data length. Mathematically, if

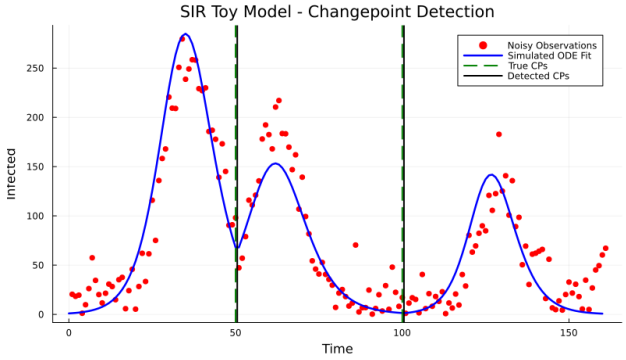

Fig 7. Application of MICA to a noisy toy dataset generated by an SIR model with two change points ~~,~~ and data length ~~150~~150. Legend: noisy observations (red dots), simulated ODE fit (blue line), true change points (green dashed vertical lines), detected change points (black solid vertical lines).

the maximum data length is D, then all change points must satisfy the condition $Change\_Point\_Position \leq D$. Figure **??** provides an example of such a dataset, generated with two change points (50, 100), 160 data points, and uniform noise. As shown in the figure, MICA successfully detects both change points.

Synthetic Benchmarking Results

Benchmarking performance is evaluated using a segment-wise loss function,

$$\mathcal{L}(\beta, \gamma) = \sum_{i=1}^{k+1} c\big(x_{t_{i-1}:t_i}, \hat{x}_{t_{i-1}:t_i}(\beta_i, \gamma)\big) + \text{pen} \tag{7}$$

where $x_{t_{i-1}:t_i}$ denotes the observed infected counts and $\hat{x}_{t_{i-1}:t_i}(\beta_i, \gamma)$ denotes the corresponding model simulations over segment i using Equations **??**. Here, β_i is a segment-specific transmission parameter allowed to vary across segments, while γ is a non-segment-specific parameter shared across all segments. The function $c(\cdot, \cdot)$ denotes the root mean square error (RMSE) computed within each segment.

The penalty term controls model complexity and is defined as

$$\text{pen} = \kappa\, p \log(n),$$

where κ is a penalty coefficient, p is the number of segment-specific model parameters, and n is the total number of data points. This formulation balances goodness of fit against model complexity, discouraging over-segmentation. ~~The same loss and penalty structure is applied consistently across all synthetic experiments~~ This κ-heuristic is applied only in the original SIR synthetic experiments reported here; the expanded toy benchmark in Section 3.2.3 and the real-world applications use the principled information-criterion penalties described in the Methods and in Supplementary Section S5.

To quantify detection performance, we compute Recall, Precision, and the F1 Score, with an emphasis on Recall as the most relevant metric for identifying all true change points. ~~Figures X–Y illustrate~~ Figure **??**(a–c) illustrates accuracy trends under Gaussian and Uniform noise. MICA consistently achieved high recall across a broad range of noise levels and penalty configurations, particularly when penalty values were appropriately tuned. Additionally, the runtime associated with each combination of controlled experimental conditions is presented in Figure ~~Z~~ **??**(d), illustrating the algorithm's computational efficiency under varying scenarios. Across all tested configurations, typical runtimes ranged from approximately 40 to 200 seconds, depending on the number of change points, time points, and noise level.

Regarding runtime, we recorded the number of numerical iterations performed by the ODE solver. Each simulation required approximately 3.7×10^4 accepted steps and 2.2×10^5 function evaluations, which represents the main numerical cost of our benchmarking. As the data length increases, more candidate

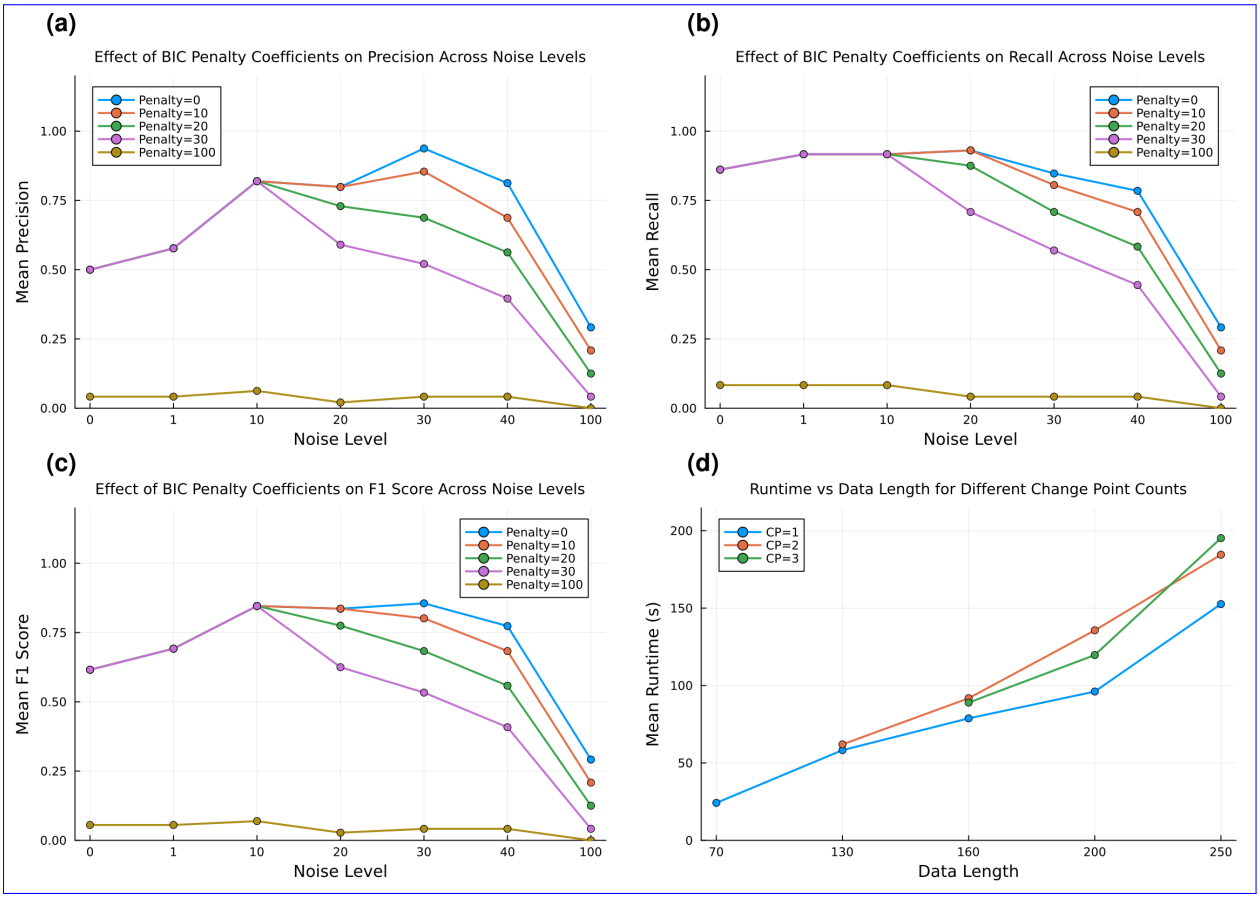

Fig 8. Synthetic SIR benchmarking results. (a) Precision, (b) recall, and (c) F1 ~~Score vs~~score across noise levels for different BIC penalty coefficients. ~~Noise Levels~~ (d) Runtime versus data length for ~~Various Penalty Coefficients Using BIC-Based Penalty Scheme~~different numbers of change points.

segments must be evaluated, and thus more simulations are required. Consequently, the overall runtime is primarily driven by the number of solver iterations accumulated across all segments, rather than by changes in the cost of individual simulations. It is important to note that these benchmarks represent a worst-case scenario, since the chosen model does not admit an analytical solution and must be integrated numerically. For models with closed-form solutions, runtimes would be substantially lower, even though the scaling with respect to data length and change points would remain the same.

~~Precision vs. Noise Levels for Various Penalty Coefficients Using BIC-Based Penalty Scheme~~
~~Recall vs. Noise Levels for Various Penalty Coefficients Using BIC-Based Penalty Scheme~~
~~runtime vs data length for different numbers of change points~~

Overall, the synthetic evaluation confirms that MICA is capable of accurately detecting change points even in noisy or short time series, provided that the penalty term is properly calibrated.

Benchmarking on the Turing Change Point Dataset

To assess MICA on real-world, model-agnostic change-point problems, we benchmarked it against the 15 published competitor algorithms from the TCPD study plus one additional HMM-Regime baseline on the Turing Change Point Dataset (TCPD) of van den Burg and Williams [?]. The TCPD comprises 42 annotated univariate time series from finance, economics, demography, environmental monitoring, and quality control; each series was annotated by multiple human experts. Following the TCPD protocol, a detected change point is counted as correct when it lies within a ±5-day window of an annotated change

point. Precision is computed against the union of all annotators, recall is averaged over the per-annotator ground-truth labels, and the reported F1 score balances the two. Full benchmark details—including the 42 datasets, the MICA model families tested, the penalty grids, the competitor baselines, and the optimization settings—are given in Supplementary Section S1.

For this benchmark, MICA was configured as a generic segmentation method using the model-informed cost functions built into `Mica.jl`—including piecewise constant, linear, quadratic, cubic, autoregressive, mean-drift, and exponential models—combined with the 19-objective TCPD-style penalty family (BIC/MDL/AIC and their TCPD, WBS, and RFPOP variants; see Supplementary Section S5). The oracle mode (MICA-O) is reported: the model family and objective are chosen per dataset to maximize F1 against the true labels. With this selection, MICA attained a mean F1 of 0.9088, the highest among all methods tested (Supplementary Table S1; Figure **??**b). The best published competitors were BOCPD (0.879), ECP (0.783), BinSeg and SegNeigh (0.781), RFPOP (0.779), and PELT (0.773). As an additional non-TCPD baseline we ran a Gaussian hidden Markov model (HMM-Regime) via the official `hmmlearn` package, selecting the number of states K by an oracle grid; its mean oracle F1 was 0.637. Representative fits and annotator labels for six TCPD datasets are shown in Supplementary Figure S1.

Benchmarking on synthetic toy datasets

In addition to the real-world TCPD benchmark, we evaluated MICA and the same competitor methods on a controlled set of synthetic toy datasets with known ground-truth change points. The three model families are:

- **ODE/SIR:** an SIR epidemic model with fixed recovery rate $\gamma = 0.7$ and segment-specific transmission rates $\beta \in \{0.00009, 0.00014, 0.00025\}$, change points at $t = 50$ and $t = 100$, and uniform observation noise at levels 0, 10 and 20 ($n = 163$).
- **Piecewise-linear regression (LR):** a continuous piecewise-linear signal with slopes $\{0.8, -0.5, 1.2, -0.8, 0.4\}$ and change points at $t = 40, 80, 120, 160$, plus Gaussian noise at levels 0, 5 and 10 ($n = 201$).
- **AR(1):** an AR(1) process with autoregressive coefficients $\{0.0, 0.9, -0.9\}$ and change points at $t = 100$ and $t = 200$, plus Gaussian observation noise at levels 0, 0.5 and 1.0 ($n = 300$).

The same 15 TCPD-paper competitor algorithms were run through their reference R and Python packages, using the oracle hyperparameter grids described in Supplementary Section S3. MICA was run with the 19-objective TCPD-style penalty family in `Mica.jl` (Supplementary Section S5); the reported MICA-O-TCPD result selects, per dataset, the objective that maximizes F1 against the true labels. Because the competitors are configured with oracle information, only oracle results are reported here.

With oracle selection, MICA attained a mean F1 of 0.8878 across the nine toy datasets, far above the best competitor (BOCPD, 0.5492; BOCPDMS, 0.4228; PELT, 0.4131). The mean oracle F1 scores are shown in Figure **??**(a); per-dataset scores are given in Supplementary Table S5.

Table **??** summarizes the advantages, limitations, and typical application scenarios of MICA and representative state-of-the-art CPD methods.

MICA in COVID-19 Modeling

We applied MICA, to COVID-19 epidemiological data from Germany spanning January 27, 2020, to March 2, 2021, hence capturing the first two waves in Germany. We employed a system of differential equations incorporating 11 compartments, each representing distinct states in the disease progression. The variable S denotes susceptible individuals at risk of infection. E_0 and E_1 represent individuals in the early and secondary latent (exposed but not yet infectious) stages, respectively. The compartment I_0 accounts for asymptomatic infected individuals, while I_1 captures symptomatic cases. Individuals requiring regular hospitalization are represented by I_2, and those admitted to intensive care units are categorized under I_3. Recovered individuals, who are assumed to have acquired immunity, are represented by R, and deceased individuals are denoted by D. In addition to these, C represents the cumulative number of infections, and

Table 3. Comparison of MICA with representative state-of-the-art change-point-detection methods.

Method	**Family**	**Advantages**	**Limitations**	**Applicable scenarios**
MICA	Model-informed optimization	Estimates CPs and parameters; interpretable shifts; UQ capable	Computationally costly; requires a user-specified model and pre-specified SP/NSP parameters	ODE, AR, and ETS models where parameter shifts are interpretable
PELT / BinSeg / SegNeigh	Dynamic programming	Fast; well-understood; no model required	Only distributional changes; no mechanistic interpretation; no UQ	Generic univariate/multivariate series with simple shifts
CPNP / WBS / ECP	Search / multivariate CPD	Robust; handles mean/variance/covariance changes	Model-free; no parameter interpretability; no CP uncertainty	Exploratory CPD on multivariate data
BOCPD / BOCPDMS	Bayesian online CPD	Probabilistic CP posterior; online inference	Parametric assumptions; no explicit dynamical model; limited parameter UQ	Online monitoring with parametric models
PROPHET	Forecasting-based CPD	Fast trend/seasonality forecasting with uncertainty	Trend changes only; no general CPD; no mechanistic model	Business/time-series forecasting with seasonality
RFPOP	Robust functional pruning	Fast robust mean-shift detection	Only mean shifts; no model structure; no parameter inference	Robust univariate CPD with outliers
HMM-Regime	State-space / regime switching	Probabilistic regime inference; can encode model structure	Requires state-space model; regimes may not map to physical parameters	Regime-switching models in finance/biology/engineering

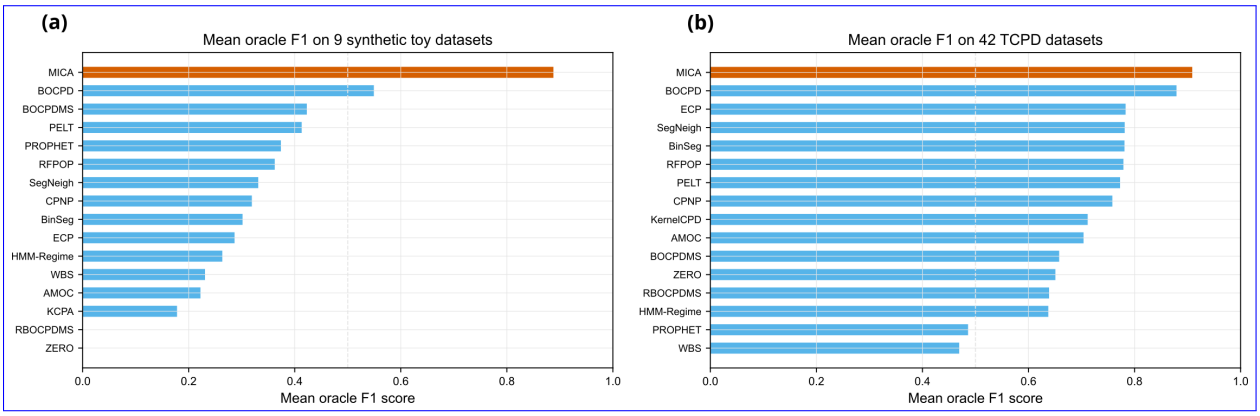

Fig 9. Mean oracle F1 scores. (a) Nine synthetic toy datasets. (b) 42 TCPD datasets. MICA selects the best model-informed cost function per dataset; competitors use their own oracle hyperparameter grids. Additional baselines are described in the main text.

V denotes the cumulative number of vaccinated individuals. This compartmental structure provides a comprehensive framework for understanding the temporal dynamics of the pandemic and assessing the impact of interventions. Table **??** summarizes the model parameters, with the final column indicating whether each parameter is Segment-Specific (SP) or Non-Segment-Specific (NSP). Segment-Specific parameters are allowed to vary across detected change points, enabling the model to adapt to temporal shifts in disease dynamics, while Non-Segment-Specific parameters remain constant across all segments.

$$
\begin{aligned}
\frac{dS}{dt} &= -\lambda(t)S + \omega R - \nu S, \\
\frac{dE_0}{dt} &= \lambda(t)S - \epsilon_0 E_0, \\
\frac{dE_1}{dt} &= \epsilon_0 E_0 - \epsilon_1 E_1, \\
\frac{dI_0}{dt} &= (1-p_1)\epsilon_1 E_1 - \gamma_0 I_0, \\
\frac{dI_1}{dt} &= p_1\epsilon_1 E_1 - \gamma_1 I_1, \\
\frac{dI_2}{dt} &= p_{12}\gamma_1 I_1 - \gamma_2 I_2, \\
\frac{dI_3}{dt} &= p_{23}\gamma_2 I_2 - \gamma_3 I_3, \\
\frac{dD}{dt} &= p_{1D}\gamma_1 I_1 + p_{2D}\gamma_2 I_2 + p_{3D}\gamma_3 I_3, \\
\frac{dR}{dt} &= \gamma_0 I_0 + (1-p_{12}-p_{1D})\gamma_1 I_1 + (1-p_{23}-p_{2D})\gamma_2 I_2 \\
&\quad + (1-p_{3D})\gamma_3 I_3 - \omega R + \nu S, \\
\frac{dC}{dt} &= p_1\epsilon_1 E_1, \\
\frac{dV}{dt} &= \nu S.
\end{aligned}
\tag{8}
$$

~~$\frac{dD}{dt} = p_{1D}\gamma_1 I_1 + p_{2D}\gamma_2 I_2 + p_{3D}\gamma_3 I_3$~~

The force of infection is defined as

$$\lambda(t) = \beta\, \delta_t\, \frac{E_1 + I_0 + I_1}{N},$$

where β denotes the baseline transmission rate and δ_t is a seasonality factor given by

$$\delta_t = 1 + \delta \cos\left(2\pi \frac{t - t_0}{365}\right).$$

Here, δ represents the amplitude of seasonal variation, t denotes time (in days) and t_0 is a reference time, set to 0 by default.

Furthermore, the total population N is given by:

$$N = S + E_0 + E_1 + I_0 + I_1 + I_2 + I_3 + R + D$$

Change points in this context reflect structural transitions, such as the impact of public health interventions, shifts in population behavior, or the emergence of viral variants. MICA extends standard compartmental models by allowing selected parameters to vary across time segments, thereby capturing the evolving dynamics of the pandemic more effectively. For example, a particular lockdown measure in schools may change contact rates and, thus, infection rates between segments, but will not affect recovery rates of infected individuals, whereas e.g. a new viral variant will affect different sets of model parameters.

An objective function is designed to assess the goodness of fit between the observed real-world data and the simulated data generated by the model. The available data includes infected cases, hospitalizations, ICU admissions, deaths, and vaccination data. On the modeling side, we utilize Equations **??** to simulate the corresponding datasets. The objective function ($\mathcal{L}$) for fitting the model is therefore formulated as follows:

$$\mathcal{L}(\Theta) = \sum_{k=1}^{K} c\big(x_{t_{k-1}:t_k}, \hat{x}_{t_{k-1}:t_k}(NSP, SP_k)\big) + \text{pen}. \tag{9}$$

Here, $x_{t_{k-1}:t_k}$ denotes the observed data and $\hat{x}_{t_{k-1}:t_k}(NSP, SP_k)$ denotes the corresponding model simulation obtained by solving Equations **??** using the non-segment-specific parameters NSP, which are shared across all segments, and the segment-specific parameter set SP_k associated with segment k, as summarized in Table **??**. The function $c(\cdot)$ denotes the mean absolute error (MAE) computed within each segment.

Because the observed channels (infections, hospitalizations, ICU admissions, deaths, and vaccinations) span very different scales, the objective is computed on log-transformed observations with equal per-channel weights. This prevents the large-magnitude vaccination and death channels from dominating the loss.

To control the number of detected segments and discourage over-segmentation, a ~~Bayesian Information Criterion (BIC)–inspired penalty~~ complexity penalty inspired by information criteria was employed. The penalty term is defined as

$$\text{pen} = \kappa\, p \log(n),$$

where p denotes the number of segment-specific parameters and n is the total number of data points. The constant ~~$\kappa = 40.0$~~ $\kappa = 40.0$ was selected empirically by evaluating a small range of values and choosing the one that produced stable and consistent segmentation results across repeated analyses. This choice was based on empirical performance rather than formal optimization.

In addition to the empirical penalty used above, we evaluated principled information-criterion penalties (BIC, MDL, and AIC) that count the total number of free parameters, including change-point locations:

$$p_{\text{total}} = n_{\text{global}} + n_{\text{segments}}\, n_{\text{segment-specific}} + n_{\text{CP}}.$$

Here n_{global} counts shared (NSP) parameters, $n_{\text{segment-specific}}$ counts parameters estimated independently in each segment, $n_{\text{segments}} = n_{\text{CP}} + 1$, and n_{CP} counts each change-point location as an estimated parameter. Table **??** summarizes how alternative balancing components (channel weights), loss transformations, and penalties affect the detected change points.

Balancing component + loss	**Penalty**	**#CPs**	**Detected CPs**
Equal-weight log loss (baseline)	zero / legacy $\kappa = 40$	8	40;60;80;120;150;260;290;320
BIC / equal-weight log loss	bic	2	60;150
MDL / equal-weight log loss	mdl	2	60;150
AIC / equal-weight log loss	aic	3	30;60;150
Inv-mean + Box–Cox	aic	6	40;60;80;120;250;320
Inv-std + Box–Cox	aic	5	40;60;80;230;280
Inv-mean + sqrt	aic	4	60;70;180;270
Inv-std + sqrt	aic	4	70;170;240;330
Norm-mean + Box–Cox	aic	2	60;80

Table 4. Sensitivity of COVID-19 detected change points to data scaling and loss transformation.

Figure **??** presents the results of change point detection obtained using MICA. The top panel compares observed and simulated data for reported infections, hospitalizations, ICU admissions, and deaths. Detected change points are marked by vertical green lines. The bottom panel shows the relative change to the first segment (RCFS) for each segment-specific parameter: detection rate (p_1), infection rate (β), hospitalization rate (p_{12}), ICU admission rate (p_{23}), and stage-specific fatality rates (p_{1D}, p_{2D}, p_{3D}).

Eight change points were detected in approximately 90 minutes, dividing the epidemic timeline into nine distinct regimes. These regimes correspond to periods marked by major policy interventions, behavioral adjustments, and epidemiological shifts, with each transition reflected in parameter dynamics. The first change point, on March 6, 2020, coincided with border closures and the cancellation of large gatherings.

Parameter	Explanation	SP/NSP
β	Transmission rate of the disease	SP
ω	Rate of loss of immunity	NSP
ν	Vaccination rate	SP
ϵ_0	Transition rate from E_0 to E_1	NSP
ϵ_1	Transition rate from E_1 to I_0	NSP
γ_0	Recovery rate from asymptomatic infection (I_0)	NSP
γ_1	Transition rate from symptomatic infection (I_1) to hospitalization or recovery	NSP
γ_2	Transition rate from regular hospitalization (I_2) to ICU hospitalization (I_3)	NSP
γ_3	Recovery rate from ICU hospitalization (I_3)	NSP
p_1	Proportion of E_1 developing symptomatic infection (I_1)	SP
p_{12}	Proportion of I_1 requiring regular hospitalization (I_2)	SP
p_{23}	Proportion of I_2 requiring ICU care (I_3)	SP
p_{1D}	Proportion of I_1 resulting in death	SP
p_{2D}	Proportion of I_2 resulting in death	SP
p_{3D}	Proportion of I_3 resulting in death	SP

Table 5. Model parameters and their classification as segment-specific (SP) or non-segment-specific (NSP).

The second, on March 26, 2020, followed nationwide lockdown measures including school closures, closure of churches, cancellation of sports events, and restrictions on non-essential shops. The third change point, on April 15, 2020, aligned with the introduction of stricter distancing measures and the Corona-Warn-App. The fourth, on May 25, 2020, corresponded to the reopening of schools and sports activities. The fifth, on June 24, 2020, coincided with broader relaxation of distancing rules and border reopening. The sixth, on October 12, 2020, was observed just prior to stay-at-home orders. The seventh, on November 11, 2020, matched the implementation of mask mandates in schools and partial lockdowns. Finally, the eighth change point, on December 11, 2020, occurred immediately before renewed school closures and full lockdowns.

For comparison, Figure ?? shows the same model fitted with the principled BIC information criterion on the raw count scale. BIC selects two change points (indices 60 and 150, approximately March 26 and June 24, 2020), which coincide with the two main waves of the first pandemic year. This sparser, principle-driven segmentation captures the dominant regime transitions while avoiding the additional breakpoints obtained with the weaker empirical penalty.

Overall, the main parameters followed expected patterns across these phases. Transmission (β) decreased sharply after the first interventions in March, falling to less than 5% of baseline after border closures and further after nationwide lockdowns. Subsequent measures in April sustained low transmission, while partial and broader relaxations from late May and June were accompanied by stable or slightly higher β, followed by a clear rebound during the summer. The renewed restrictions in autumn again suppressed transmission, and the December lockdown brought β to its lowest recorded level (around 2% of baseline). Detection (p_1) also tracked intervention phases. It rose strongly after the early March measures, reflecting rapid expansion of testing, dipped somewhat after the March 26 lockdown, and declined further into April. With reopening from May onward, detection recovered, reached higher levels during the summer, and peaked in October at over eight times the baseline. Detection remained elevated through the subsequent restrictions in November and December. Hospitalization (p_{12}) and ICU admission (p_{23}) probabilities showed more modest variation but were broadly consistent with intervention timing. These parameters were stable during the first lockdown, declined in April, and then increased with the partial and broader reopening phases. After October, hospitalization returned to baseline values, while ICU admission fell again under the later restrictions. Fatality parameters (p_{1D}, p_{2D}, p_{3D}) showed more variability across phases. In general, mortality ratios decreased relative to baseline after the first interventions, rose during certain later phases, and remained elevated in December despite strong suppression of transmission.

While these broad trends align with expectations, not all parameter shifts followed the anticipated pattern. For example, infection and hospital fatality occasionally increased even during phases when transmission was falling or detection was rising, and ICU fatality sometimes moved in the opposite direction from the other mortality measures. Beyond the model's internal mechanics, these patterns might

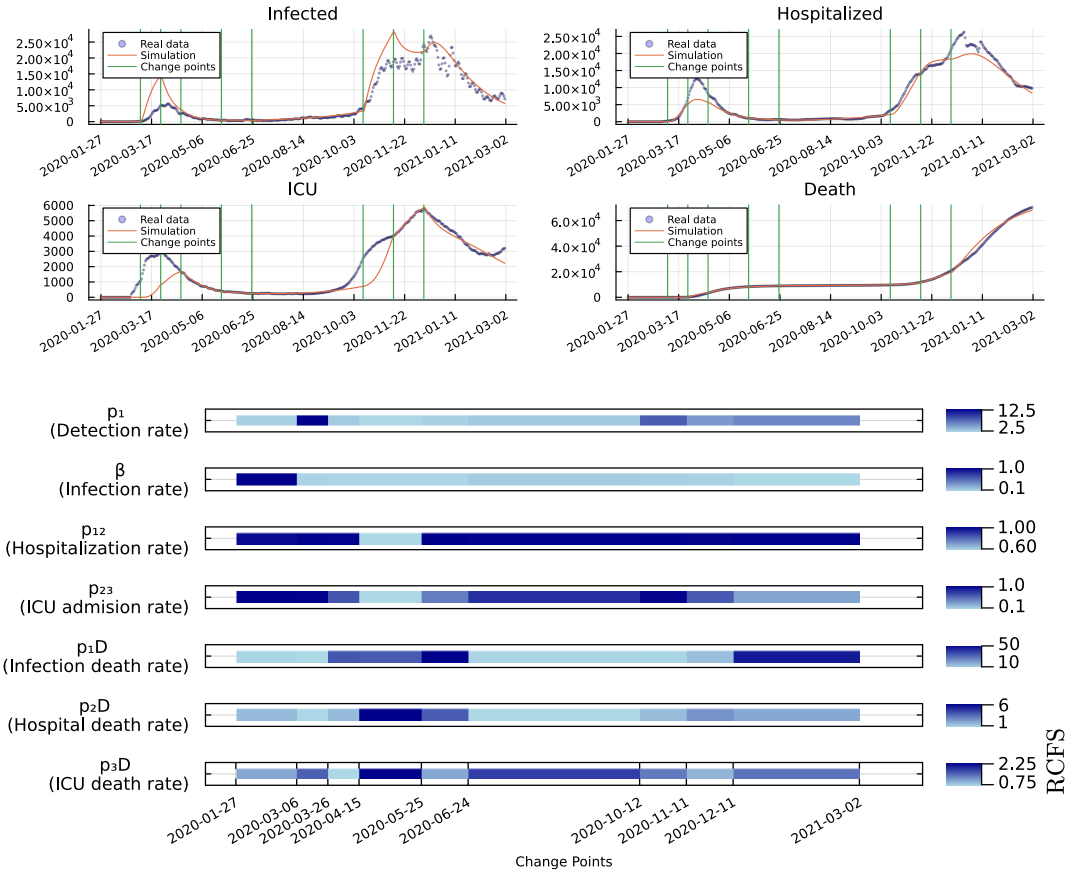

Fig 10. Simulated COVID-19 dynamics in Germany with detected change points and segment-specific parameter shifts. Top: Observed vs. simulated infections, hospitalizations, ICU admissions, and deaths. Vertical green lines indicate detected change points. Bottom: Relative change of segment-specific parameters with respect to the first segment (RCFS), revealing shifts in transmission, hospitalization, and fatality dynamics over time.

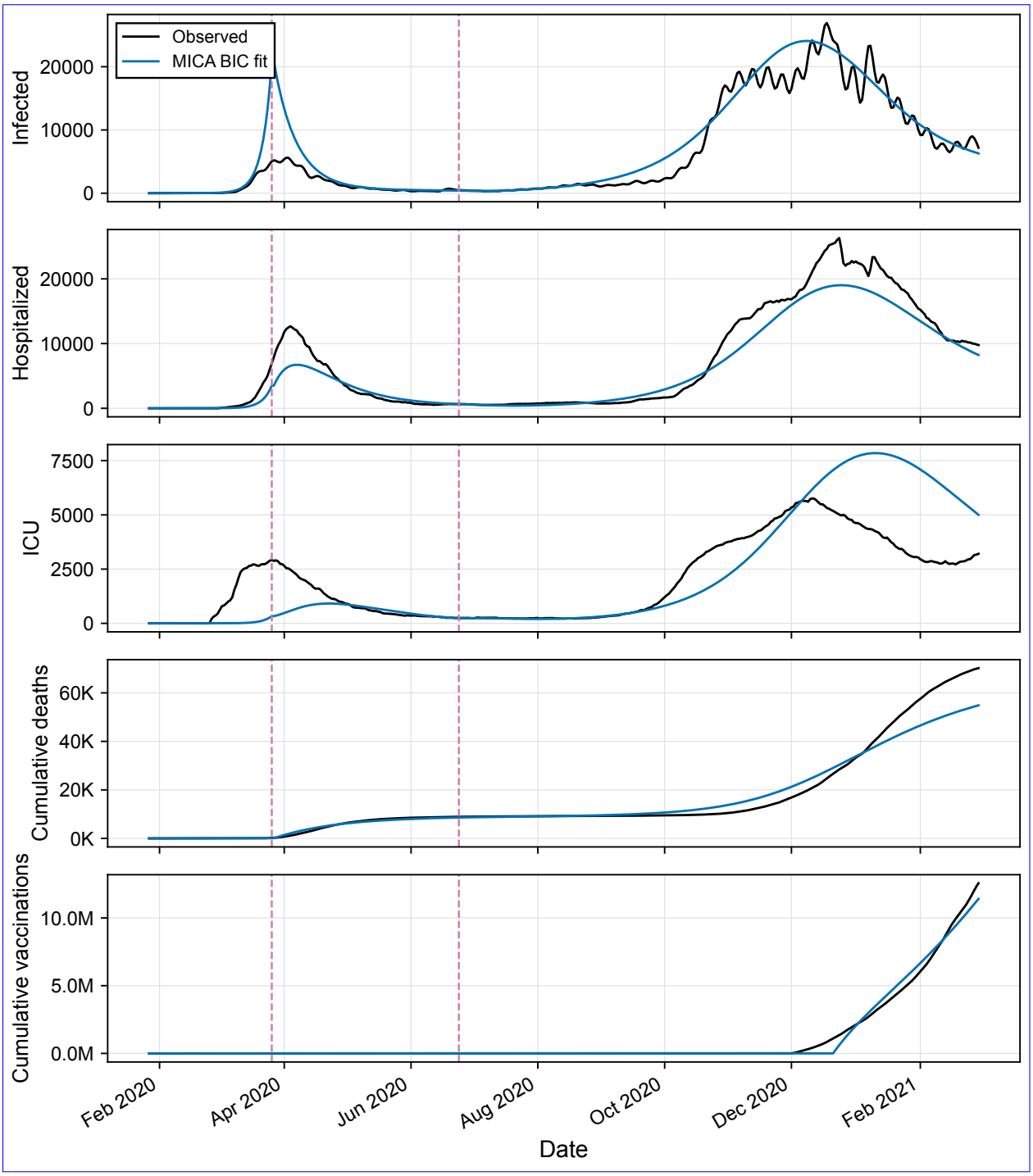

Fig 11. COVID-19 dynamics in Germany fitted with MICA using the BIC information criterion, shown on the raw count scale. BIC selects two change points at indices 60 and 150 (approximately March 26 and June 24, 2020; green dashed vertical lines), capturing the two main epidemic waves. Observed data are shown in blue and the MICA simulation in orange.

also reflect real-world processes such as reporting delays, changes in who was being tested, or variation in the severity of illness over time. A random change-point baseline comparison is provided in Supplementary

Section S6; the MICA BIC solution achieves a substantially lower loss than random CP placements with the same or larger numbers of change points. Profile-likelihood confidence intervals for the BIC two-change-point solution are reported in Supplementary Section S7; 17 of 32 parameters and both change points are practically identifiable under the assumed L2 Gaussian likelihood.

MICA in Wind Turbine Monitoring

Wind turbines operate under variable environmental and load conditions, making early detection of internal behavioral shifts essential for reliability and predictive maintenance. While status logs record events like start-ups and shutdowns, they offer limited insight into how such conditions impact internal dynamics, particularly generator cooling behavior influenced by wind speed, ambient temperature, and thermal resistance. In this study, we apply MICA to thermal model parameters to identify abrupt changes in cooling dynamics that may indicate operational faults or emerging component degradation. By aligning detected change points with turbine status events, we aim to enhance fault diagnostics and enable more informed maintenance strategies.

Several recent studies have explored fault detection in wind turbines using Supervisory Control and Data Acquisition (SCADA) data, employing approaches such as thermophysics-based diagnostics and kernel-based statistical methods [**?**, **?**, **?**]. While these methods provide valuable insights, they often lack adaptability in model structure, particularly in supporting dynamic parameter updates or accommodating global parameter consistency alongside local variability. Many rely heavily on the statistical properties of the data without integrating mechanisms for dynamic model refinement. In contrast, MICA addresses this gap by enabling selective parameter adaptation across temporal segments: certain parameters are held constant to maintain global stability, while others are allowed to vary to capture local behavioral changes. This hybrid structure improves both modeling flexibility and fault detection accuracy in operational settings.

The Kelmarsh Wind Farm SCADA dataset was used as the second application of MICA. The dataset was released by Cubico Sustainable Investments Ltd. under a CC-BY-4.0 license and exported via the Greenbyte platform on January 27, 2022 [**?**]. The dataset comprises 10-minute resolution SCADA and event log data from six Senvion MM92 turbines in the UK, covering the period from 2016 to mid-2021. For this study, we focused on a single turbine, Kelmarsh 1, over an 18-day window from January 1 to January 18, 2021. The analysis used three primary signals, generator temperature, wind speed, and ambient temperature, all recorded at 10-minute intervals. These were used as inputs to the thermal model and change point detection framework.

The mathematical model employed in this application is based on the "Model Sensor" framework by Zhang et al. [**?**], which captures the dynamic relationship between generator temperature and environmental inputs. The key idea is to estimate generator temperature based on energy balance principles, accounting for copper losses and heat dissipation via thermal resistance. Figure **??** illustrates this conceptual model.

The model captures the relationship between the generator temperature T_g, wind speed V_w, and ambient temperature T_a. This relationship is governed by the energy balance equation [**?**], which considers the energy input Q_{in} due to copper losses and the energy output Q_{out} due to cooling:

$$Q = Q_{\text{in}} - Q_{\text{out}}$$

The generator winding temperature change (ΔT) is related to the energy Q and thermal capacitance C_g as [**?**]:

$$Q = C_g \Delta T_g = C_g \left(T_g(k) - T_g(k-1)\right)$$

where $T_g(k)$ represents the generator winding temperature at time k.

It is considered that the nonlinear relationship between copper losses and wind speed V_w, which can be approximated by a third-degree polynomial [**?**]:

$$Q_{\text{in}} = f(V_w) = f_3 V_w^3 + f_2 V_w^2 + f_1 V_w$$

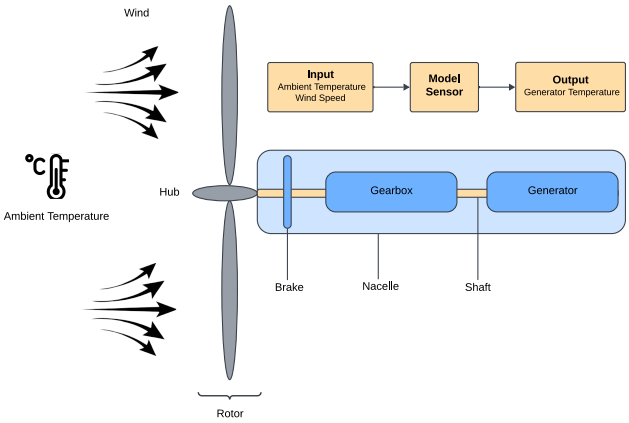

Fig 12. This figure depicts the key components of a wind turbine, including the blades, gearbox, yaw system, and generator winding. The focus is on the relationship between the generator temperature (model output) and two model inputs: ambient temperature and wind speed. The "Model Sensor" illustrates the dynamic relationship between these inputs and the output, highlighting how changes in the ambient conditions and wind speed affect the generator's temperature.

Similarly, the heat conduction due to cooling, Q_{out}, is modeled considering the thermal resistance R_{ga} [**?**], which depends on wind speed through another third-degree polynomial [**?**]:

$$Q_{\text{out}} = \frac{T_g - T_a}{R_{ga}}$$

where

$$R_{ga} = g(V_w) = g_3 V_w^3 + g_2 V_w^2 + g_1 V_w + g_0$$

The generator temperature is thus given by:

$$T_g(k) = \frac{C_g^{-1} f(V_w(k)) + T_g(k-1) - T_a(k)}{1 + C_g^{-1} t_s g(V_w(k))} + T_a(k)$$

By substituting $f(V_w(k))$ and $g(V_w(k))$ from previous equations, we obtain the final model that describes the relationship between generator temperature, ambient temperature, and wind speed. This model is expressed as:

$$y(k) = \frac{\theta_1 u_1^3(k) + \theta_2 u_1^2(k) + \theta_3 u_1(k) + y(k-1)}{\theta_4 u_1^3(k) + \theta_5 u_1^2(k) + \theta_6 u_1(k) + \theta_7} - \frac{u_2(k)}{\theta_4 u_1^3(k) + \theta_5 u_1^2(k) + \theta_6 u_1(k) + \theta_7} + u_2(k) \tag{10}$$

where $u_1 = V_w^*$ and $u_2 = T_a^*$ are the model inputs, and $y = \hat{T}_g$ is the model output. And the parameter vector $\boldsymbol{\theta}$ is defined as:

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \\ \theta_5 \\ \theta_6 \\ \theta_7 \end{bmatrix} = C_g^{-1} \begin{bmatrix} f_3 \\ f_2 \\ f_1 \\ t_s g_3 \\ t_s g_2 \\ t_s g_1 \\ C_g + t_s g_0 \end{bmatrix}$$

where $\boldsymbol{\theta}$ is the vector of model sensor parameters that need to be updated regularly for wind turbine change point detection.

We classify model parameters into segment-specific (SP) and non-segment-specific (NSP) types, as detailed in Table **??**. Parameters related to copper losses, specifically those governing the relationship between wind speed and electrical heating, are assumed to be constant across all segments. This reflects the assumption that electrical losses due to current flow in the windings remain relatively stable regardless of operational changes or fault conditions. In contrast, parameters associated with thermal resistance are treated as segment-specific, allowing the model to adapt to varying cooling dynamics influenced by operational or environmental changes. For example, cooling efficiency may vary with fluctuations in wind speed, ambient temperature, or turbine state (e.g., during startup or fault recovery). Allowing thermal resistance parameters to vary across segments enhances the model's ability to capture these dynamic effects and improves the precision of change point detection. The full classification of parameters is summarized in Table **??**, where SP denotes segment-specific parameters and NSP denotes those held constant across the entire time series.

Physically, the parameters have the following interpretation. The coefficients θ_1, θ_2, and θ_3 describe how electrical (copper) losses in the generator windings increase with wind speed; higher wind speeds generally mean higher generator loading and therefore more heat generation. Because the electrical properties of the windings are unlikely to change over short observation windows, these coefficients are treated as global (NSP). The coefficients θ_4, θ_5, and θ_6 describe how the thermal resistance between the generator and the ambient air depends on wind speed; they capture cooling efficiency and are allowed to vary across segments (SP) because cooling conditions can change with turbine state, ambient conditions, or faults. The parameter θ_7 combines the generator heat capacity with a baseline thermal-resistance term and governs the overall thermal inertia and baseline cooling level; it is also allowed to vary across segments (SP).

To estimate the parameters of the model in Equation **??**, we define the following objective function:

$$\mathcal{L}(\theta) = \sum_{s} \sum_{k=1}^{N_s} \left(T_g^*(k) - \hat{T}_g(k, \theta_g, \theta_s) \right)^2 + \text{pen}. \tag{11}$$

Here, $\hat{T}_g(k, \theta_g, \theta_s)$ denotes the model-predicted generator temperature obtained by solving Equation **??** at time point k, where θ_g represents the non-segment-specific parameters and θ_s denotes the segment-specific parameters associated with segment s, as detailed in Table **??**. The quantity $T_g^*(k)$ denotes the observed generator temperature from the SCADA dataset at time point k, and N_s is the number of observations within segment s.

The penalty term in this application followed a ~~BIC-based~~ complexity-penalty formulation to regulate the number of detected segments and to mitigate over-segmentation. Specifically, we used

$$\text{pen} = \kappa p \log(n),$$

where p is the number of segment-specific parameters and n denotes the data length. The constant $\kappa = 100.0$ was determined empirically after testing several values and selecting the one that produced stable and interpretable segmentations across repeated runs. This choice was made through empirical assessment rather than formal optimization.

Figure **??** presents the ~~results of thermal modeling and change point detection using MICA. The top subplot shows detected change pointssuperimposed on the actual and simulated generator temperatures. The bottom subplot illustrates the relative change in segment-specific parameters (θ_1–θ_4) compared to the~~

Parameter Name	Parameter Description	Type
θ_1	Coefficient related to copper loss polynomial	NSP
θ_2	Coefficient related to copper loss polynomial	NSP
θ_3	Coefficient related to copper loss polynomial	NSP
θ_4	Coefficient related to thermal resistance polynomial	SP
θ_5	Coefficient related to thermal resistance polynomial	SP
θ_6	Coefficient related to thermal resistance polynomial	SP
θ_7	Coefficient related to thermal resistance polynomial	SP

Table 6. Model parameters and classification into segment-specific (SP) and non-segment-specific (NSP)

~~first segment. These parameters represent thermal resistance coefficients and reflect changes in the cooling dynamics of the generator~~wind-turbine result obtained with the BIC, MDL, and AIC information criteria. All three criteria selected the same four change points, indicating a stable segmentation. The panel shows the observed generator-bearing temperature, the MICA simulation, and the detected change points.

~~Fifteen~~ Four change points were detected ~~within approximately 34 minutes of computation for a data length of 2, 500 time points. The first change point (~~at 2021-01-01 ~~16~~23~~:30) coincides with a startup sequence involving mains run-up and gearbox warm-up. This is marked by a sharp drop in θ_1 and θ_2 and a simultaneous increase in θ_3 and θ_4, indicating a transition~~ 20, 2021-01-04 11:20, 2021-01-08 23:40, and 2021-01-12 22:00. The first captures the thermal transition during the startup sequence on 2021-01-01, as the turbine moves from stationary to active thermal ~~behavior. Several subsequent change points (e.g.,~~ behaviour. The second marks a regime shift on 2021-01-04~~07:50, 2021-01-05 17:10) occur in the absence of status log entries but feature spikes in θ_4, suggesting undocumented or latent thermal shifts.~~

~~Other change points align with known events such as external stops due to low wind (e.g., 2021-01-07 07:30~~, shortly after a data-communication interruption. The third falls within a period of low-wind external stops around 2021-01-08~~01:50) and icing conditions (e.g., 2021-01-09 12:50). These typically show transient increases in θ_2 and θ_3 and varying responses in θ_4, reflecting changes in heat dissipation and cooling response.~~

~~The final segments exhibit more stable behavior, with θ parameters returning closer to baseline or oscillating within narrower ranges. However, certain transitions (e.g., 2021-01-12 12:30) show extreme drops in multiple parameters, possibly due to short-term signal artifacts or local data irregularities.~~ . The fourth coincides with the external-stop and restart cycle on 2021-01-12. Each of these four transitions has a direct counterpart in the SCADA status logs, whereas the additional eleven change points reported in the original 15-CP analysis were predominantly weakly or not aligned with logged events.

~~Overall, the alignment of parameter shifts with turbine eventssupports the model's capacity to detect and explain operational transitions. The relative parameter changes provide interpretable physical meaning, indicating when and how the generator's thermal response is altered by external conditions or internal faults.~~

The detected change points were systematically compared with turbine status logs contained in the SCADA dataset ~~to assess their alignment with known operational events. As shown in Table??, several change points closely aligned with startupsequences, including run-up, mains connection, and gearbox warm-up stages, indicating a strong correspondence between physical thermal transitions and externally reported events. Others occurred shortly before~~ (Table **??**). The four BIC/MDL/AIC change points correspond to the strongest operational transitions: startup, a data-communication-related thermal transition, low-wind external stops, ~~particularly due to low wind conditions, suggesting early indicators of shutdown behavior. A subset of change points exhibited no clear status association, highlighting either undocumented anomalies or latent physical transitionsnot captured by the status system~~and an external-stop/restart cycle. The remaining eleven change points from the weaker-penalty solution are retained in Supplementary Section S4 (original 15-CP figure and alignment table) as a sensitivity analysis; most of these had weak or no SCADA alignment and are therefore interpreted as over-segmentation artifacts rather than confirmed physical transitions. This analysis ~~supports the hypothesis thatchange point detection applied to thermal model parameters can effectively identify both recorded and unrecorded shifts in operational behaviors~~shows that, with a principled information-criterion penalty, MICA recovers

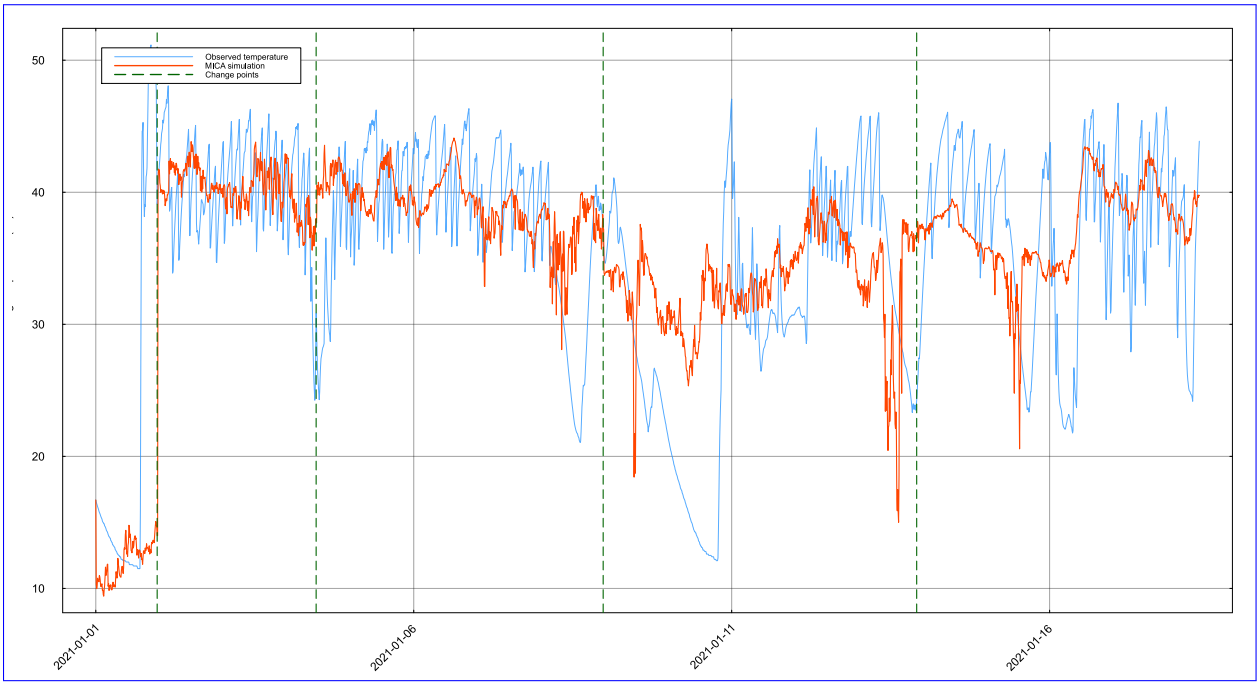

Fig 13. ~~Detected change points and their effect on~~ Wind-turbine generator ~~thermal~~ front-bearing temperature fitted with MICA using BIC/MDL/AIC model ~~parameters~~selection. ~~Top: actual~~ Observed temperature (blue), MICA simulation (orange), and ~~simulated temperatures with~~ the four detected change points (green dashed vertical lines). ~~Bottom: relative shifts in segment-specific thermal parameters across segments~~The BIC, MDL, and AIC criteria all returned the same four change points.

the major validated operational transitions while avoiding spurious breakpoints.

Change Point	Nearest Status Event	Alignment	Event Type	Description
2021-01-01 23:20	2021-01-01 16:46	Strong	Startup sequence	Captures the thermal transition during/after the full startup sequence (mains run-up and gearbox warm-up).
2021-01-04 11:20	2021-01-03 22:20	Moderate	Data communication unavailable	Thermal regime shift following a data-communication interruption recorded 13 h earlier.
2021-01-08 23:40	2021-01-08 03:27	Moderate	External stop (low wind)	Change point within a low-wind shutdown period; reflects altered cooling dynamics during operational cycling.
2021-01-12 22:00	2021-01-12 15:42	Moderate	External stop (low wind)	Change point follows an external-stop/restart cycle by 6.5 h, marking a thermal-regime shift.

Table 7. Alignment between ~~detected~~ the four BIC/MDL/AIC change points and turbine status events.

Discussion

The results presented in this study demonstrate the effectiveness and versatility of MICA, our proposed change point detection algorithm, across a variety of domains. By enabling selective parameter adaptation, MICA allows dynamic models to incorporate structural changes in a principled and interpretable manner. Applied to both synthetic datasets and real-world systems, including epidemiological modeling of COVID-19 and fault detection in wind turbine cooling systems, MICA successfully identified significant temporal shifts aligned with known events or operational anomalies.

A central strength of MICA lies in its ability to distinguish between segment-specific and non-segment-specific parameters. This feature offers a major advantage over conventional approaches that

assume either full parameter constancy or complete variability. In the COVID-19 modeling scenario, this allowed the model to adapt to shifts in disease transmission while preserving invariant biological parameters. In the wind turbine application, it enabled detection of cooling efficiency changes without incorrectly attributing noise to stable electrical losses.

Compared to existing change point detection methods, such as kernel-based techniques, fixed-threshold statistical models, or manually defined intervention points, MICA introduces a flexible and model-aware strategy. It integrates seamlessly with dynamic systems and enhances both the accuracy and explanatory power of simulation results.

Related work has developed model-free early-warning indicators for impending critical transitions. Critical slowing down, for example, monitors rising variance, autocorrelation, and recovery time as a system approaches a bifurcation [?, ?]. Dynamical network biomarkers identify small groups of variables that become strongly correlated and highly fluctuating before a transition [?]. These approaches are valuable when the underlying mechanism is unknown or too complex to model. MICA does not replace them as a generic early-warning tool; rather, it complements them. A model-free signal can flag that a transition is likely, and MICA can then be used to locate the transition precisely and attribute it to specific parameter changes, such as a shift in transmission rate or cooling efficiency.

Nevertheless, the method has ~~certain limitations . The~~ important limitations that should guide its use and interpretation. First, MICA is a heuristic, model-informed segmentation framework rather than a procedure with finite-sample statistical guarantees. The greedy forward-backward binary segmentation combined with the genetic-algorithm (or other metaheuristic) parameter optimizer does not guarantee convergence to a global optimum, and the detected change points are conditional on the user-specified mechanistic model, the chosen loss function, the scaling and normalization of the data, and the information criterion or κ-penalty used to stop the search. Consequently, a detected change point should be interpreted as "the data are better explained by allowing the parameters to change at this time" under the assumed model and criterion, not as a statistically proven structural break; MICA does not provide p-values, exact confidence sets, or calibrated hypothesis tests for the presence of a change point. Second, the selection of the penalty term significantly influences the granularity of detected change points; inappropriate tuning can lead to either over-segmentation or missed transitions. ~~. Moreover~~Third, model misspecification can produce misleading breakpoints: if the assumed model cannot capture the true dynamics, MICA may place change points that compensate for structural errors rather than for genuine parameter shifts. Fourth, alignment with real-world event logs (such as SCADA status files or policy intervention calendars) may be imperfect due to sensor delays, missing data, or asynchronous reporting~~. Additionally~~, so such alignments are descriptive rather than causal. Fifth, the current ~~use of a binary segmentation introduces a greedy search heuristic, which, while efficient, may not guarantee globally optimal segmentation. Future work will focus on improving robustness under uncertainty , supporting a broader class of dynamic models, and incorporating more general, non-greedy segmentationstrategies to enhance detection accuracy and flexibility~~implementation is designed for offline, batch analysis and scales with the cost of repeated model simulations across segments; it is therefore not suited to real-time streaming inference or to very high-dimensional state spaces without further algorithmic approximation. For users who require uncertainty intervals, we provide a separate profile-likelihood engine (`BreakpointProfiles.jl`) that reports approximate confidence intervals for parameters and change-point locations conditional on the detected segmentation; see Supplementary Section S7. These intervals rely on asymptotic arguments or bootstrap calibration and should be interpreted with the same caveats.

Despite these challenges, MICA shows strong potential for generalization. Its domain-agnostic formulation makes it applicable to a wide range of time-dependent systems, including industrial monitoring, biomedical signal processing, environmental sensing, and econometrics. Future directions include the development of automated penalty tuning methods, real-time streaming implementations, and integration with more complex or multivariate system models.

Overall, MICA provides a flexible, interpretable, and computationally viable framework for structural change detection in dynamic systems. Its capacity to operate across domains while respecting the inherent structure of underlying models positions it as a valuable tool for both scientific investigation and practical decision-making.

Conclusion

We presented MICA, a flexible and model-driven algorithm for detecting change points in time series governed by dynamic systems. Unlike traditional statistical approaches, MICA operates directly on the model's structure by allowing selective parameter segmentation, capturing changes in system dynamics while preserving parameter stability where appropriate.

By combining a tailored binary segmentation strategy with an adaptive optimization procedure, MICA enables precise and interpretable identification of structural shifts in both synthetic and real-world datasets. Its application to COVID-19 epidemiological modeling and wind turbine operational monitoring illustrates its effectiveness in revealing meaningful change points aligned with known interventions and operational events.

The algorithm's generality, demonstrated across domains with distinct system characteristics, highlights its utility for a wide range of applications, from public health modeling to industrial fault detection. As a domain-agnostic tool with built-in model awareness, MICA contributes a novel, versatile approach to structural change analysis in dynamic systems.

Acknowledgments

We thank the members of the Kaderali group, in particular Dr. Rahul Brahma and Yasas Wijesekara, for their valuable comments and suggestions during the preparation of this study. We also thank Johannes Apelt for his input on the development of the method package in Julia. Their feedback greatly contributed to improving the quality of this work.

Declarations

Funding

This study received no external funding.

Competing Interests

The authors declare no financial or non-financial competing interests.

Author Contributions

~~ML developed the methodology, implemented the computational framework, carried out the analyses, and drafted the manuscript.~~ Conceptualization: ML, LK. Data curation: ML. Formal analysis: ML. Funding acquisition: None. Investigation: ML. Methodology: ML, LK. Project administration: LK. Resources: ML. Software: ML. Supervision: LK. Validation: ML. Visualization: ML. Writing – original draft: ML. Writing – review & editing: ML, LK~~conceived the study, contributed to the design of the methodology, guided data interpretation, and contributed to manuscript preparation~~. Both authors ~~reviewed, edited,~~ read and approved the final ~~version of the~~ manuscript.

Data Availability

The wind turbine modeling data used in this study are publicly available via Zenodo at `https://doi.org/10.5281/zenodo.5841834`. The COVID-19 epidemiological datasets were obtained from publicly accessible repositories maintained by the Robert Koch Institute (RKI), Germany, including COVID-19 mortality data (`https://github.com/robert-koch-institut/COVID-19-Todesfaelle_in_Deutschland`), seven-day incidence data (`https://github.com/robert-koch-institut/COVID-19_7-Tage-Inzidenz_in_Deutschland`), hospitalization data

(https://github.com/robert-koch-institut/COVID-19-Hospitalisierungen_in_Deutschland), ICU capacity and occupancy data (https://github.com/robert-koch-institut/Intensivkapazitaeten_und_COVID-19-Intensivbettenbelegung_in_Deutschland), and vaccination data (https://github.com/robert-koch-institut/COVID-19-Impfungen_in_Deutschland). The full source code and implementation used in this study are openly available at https://github.com/Mehdilotfi7/MICA.

References

1. Page ES. Continuous inspection schemes. Biometrika. 1954;41(1/2):100-15.
2. Page E. A test for a change in a parameter occurring at an unknown point. Biometrika. 1955;42(3/4):523-7.
3. Truong C, Oudre L, Vayatis N. Selective review of offline change point detection methods. Signal Processing. 2020;167:107299.
4. Lee Y, Kim S, Oh H. Sequential change-point detection in time series models with conditional heteroscedasticity. Economics Letters. 2024;236:111597.
5. Ondrus M, Cribben I. fabisearch: A package for change point detection in and visualization of the network structure of multivariate high-dimensional time series in R. Neurocomputing. 2024;578:127321.
6. Scott AJ, Knott M. A cluster analysis method for grouping means in the analysis of variance. Biometrics. 1974:507-12.
7. Gong Y, Dong X, Zhang J, Chen M. Latent evolution model for change point detection in time-varying networks. Information Sciences. 2023;646:119376.
8. Cui Y, Yang J, Zhou Z. State-domain change point detection for nonlinear time series regression. Journal of Econometrics. 2023;234(1):3-27.
9. Shishavan HH, Gossett E, Bi J, Henning R, Cherniack M, Kim I. Unsupervised Bayesian change point detection model to track acute stress responses. Biomedical Signal Processing and Control. 2024;95:106415.
10. Janczura J, Barszcz T, Zimroz R, Wyłomańska A. Machine condition change detection based on data segmentation using a three-regime, α-stable Hidden Markov Model. Measurement. 2023;220:113399.
11. Dass SC, Kwok WM, Gibson GJ, Gill BS, Sundram BM, Singh S. A data driven change-point epidemic model for assessing the impact of large gathering and subsequent movement control order on COVID-19 spread in Malaysia. PloS one. 2021;16(5):e0252136.
12. Gu J, Yin G. Bayesian SIR model with change points with application to the Omicron wave in Singapore. Scientific Reports. 2022;12(1):20864.
13. Jiang S, Zhou Q, Zhan X, Li Q. BayesSMILES: Bayesian segmentation modeling for longitudinal epidemiological studies. Journal of Data Science. 2021;19(3):365-89. doi:10.6339/21-JDS1009.
14. van den Burg GJJ, Williams CKI. An Evaluation of Change Point Detection Algorithms. arXiv preprint arXiv:200306222. 2020. Available from: https://arxiv.org/abs/2003.06222.
15. Qiu Y, Feng Y, Sun J, Zhang W, Infield D. Applying thermophysics for wind turbine drivetrain fault diagnosis using SCADA data. IET Renewable Power Generation. 2016;10(5):661-8.
16. Letzgus S. Change-point detection in wind turbine SCADA data for robust condition monitoring with normal behaviour models. Wind Energy Science Discussions. 2020;2020:1-29.

17. Huang Y, Liu S, Chen H, Dai J, Wang X, Tao H. Dynamic behavior analysis method for the normal and faulty drivetrain of wind turbine under multi-working conditions. Energy Science & Engineering. 2023;11(10):3619-40.

18. Effenberger N, Ludwig N. A collection and categorization of open-source wind and wind power datasets. Wind Energy. 2022;25(10):1659-83.

19. Zhang S, Lang ZQ. SCADA-data-based wind turbine fault detection: A dynamic model sensor method. Control Engineering Practice. 2020;102:104546.

20. Aglen O. Loss calculation and thermal analysis of a high-speed generator. In: IEEE International Electric Machines and Drives Conference, 2003. IEMDC'03.. vol. 2. IEEE; 2003. p. 1117-23.

21. Esfandiari RS, Lu B. Modeling and analysis of dynamic systems. CRC press; 2010.

22. Tamura J. Calculation method of losses and efficiency of wind generators. Wind energy conversion systems: Technology and trends. 2012:25-51.

23. Shin D, Chung SW, Chung EY, Chang N. Energy-optimal dynamic thermal management: Computation and cooling power co-optimization. IEEE Transactions on Industrial Informatics. 2010;6(3):340-51.

24. Scheffer M, Bascompte J, Brock WA, Brovkin V, Carpenter SR, Dakos V, et al. Early-warning signals for critical transitions. Nature. 2009;461(7260):53-9. doi:10.1038/nature08227.

25. Scheffer M, Carpenter SR, Lenton TM, Bascompte J, Brock W, Dakos V, et al. Anticipating critical transitions. Science. 2012;338(6105):344-8. doi:10.1126/science.1225244.

26. Chen L, Liu R, Liu ZP, Li M, Aihara K. Detecting early-warning signals for sudden deterioration of complex diseases by dynamical network biomarkers. Scientific Reports. 2012;2:342. doi:10.1038/srep00342.

27. Lotfi M, Kaderali L. Mica documentation; 2025. Accessed July 2025. `https://changepointdetection.com`.

28. Lotfi M, Kaderali L. Mica GitHub repository; 2025. Accessed July 2025. `https://github.com/Mehdilotfi7/Mica`.